ASCENCIA KEYCE

CAMPUS DE LOMÉ

Théorie des graphes

Niveau : Master 1 – IT

K. E. HOVI

Lomé, le 10 novembre 2026

Sommaire

PartieI – COURS

1 Introduction à la théorie des graphes 3
2 Propriétés fondamentales des graphes 13
3 Les algorithmes sur les graphes 22

PartieII – TRAVAUX DIRIGÉS

1 Introduction à la théorie des graphes 37
2 Propriétés fondamentales des graphes 40
3 Les algorithmes sur les graphes 45

Table des matières

Partie I

COURS

Chapitre 1

Introduction à la théorie des graphes

Sommaire

1.1 Définitions et premiers exemples 3
1.2 Représentation des graphes 8

Introduction

La théorie des graphes trouve ses origines avec les travaux d'Euler au XVIIIe siècle, notamment avec le problème des ponts de Königsberg. D'autres problèmes, comme la marche du cavalier ou le coloriage de cartes, ont également contribué à son développement. Au XXe siècle, grâce à des mathématiciens tels que Königsberg, Menger, Cayley, Berge et Erdös, cette théorie est devenue une branche des mathématiques à part entière.
Un graphe représente les relations entre les éléments d'un ensemble complexe, et il est utilisé dans de nombreux domaines : réseaux de communication, réseaux routiers, biologie, etc. Aujourd'hui, les informaticiens utilisent la théorie des graphes pour résoudre des problèmes complexes, en particulier ceux liés à l'aspect algorithmique.

1.1 Définitions et premiers exemples

Définition 1.1.1

Un *graphe fini* $G = (V,E)$ est défini par :

- un ensemble fini $V = \{v_1,v_2,\ldots,v_n\}$, dont les éléments sont appelés *sommets* (*Vertices* en anglais) ;
- un ensemble fini $E = \{e_1,e_2,\ldots,e_m\}$, dont les éléments sont appelés *arêtes* (*Edges* en anglais).

Chaque arête $e \in E$ est définie comme une paire non ordonnée de sommets, appelée les *extrémités* de e. Si une arête e relie deux sommets a et b, alors :

- a et b sont dits *adjacents* ou *incidents avec* e ;
- l'arête e est dite *incidente* avec les sommets a et b.

Le *nombre de sommets* n d'un graphe est appelé l'*ordre* de ce graphe.

Définition 1.1.2

- **L'ordre** d'un graphe est le nombre de ses sommets.
- Une **boucle** est un arc ou une arête reliant un sommet à lui-même.
- Un graphe non-orienté est dit **simple** s'il ne comporte pas de boucle, et s'il ne comporte jamais plus d'une arête entre deux sommets. Un graphe non orienté qui n'est pas simple est un **multi-graphe**. Dans le cas d'un multi-graphe, A

n'est plus un ensemble mais un multi-ensemble d'arêtes. On se restreindra généralement dans la suite aux graphes simples.

- Un graphe orienté est un p-**graphe** s'il comporte au plus p arcs entre deux sommets. Le plus souvent, on étudiera des **1-graphes**.
- Un **graphe partiel** d'un graphe orienté ou non est le graphe obtenu en supprimant certains arcs ou arêtes.
- Un **sous-graphe** d'un graphe orienté ou non est le graphe obtenu en supprimant certains sommets et tous les arcs ou arêtes incidents aux sommets supprimés.
- Un graphe orienté est dit **élémentaire** s'il ne contient pas de boucle.
- Un graphe orienté est dit **complet** s'il comporte un arc (s_i,s_j) et un arc (s_j,s_i) pour tout couple de sommets différents $s_i,s_j \in S^2$. De même, un graphe non-orienté est dit complet s'il comporte une arête (s_i,s_j) pour toute paire de sommets différents $s_i,s_j \in S^2$.

Remarque 1.1.1

Un graphe est une structure mathématique utilisée pour modéliser des relations entre des objets. Il est défini comme suit :

- **Sommet (ou nœud) :** Un point du graphe qui représente un objet ou une entité.
- **Arête :** Une connexion entre deux sommets, représentant une relation.
- **Degré :** Le nombre d'arêtes incidentes à un sommet.

Pour les graphes dirigés, on distingue le degré entrant et le degré sortant.

1.1.1 Notion d'adjacence entre sommets

- Dans un graphe non orienté, un sommet s_i est dit **adjacent** à un autre sommet s_j s'il existe une arête entre s_i et s_j. L'ensemble des sommets adjacents à un sommet s_i est défini par :
$$adj(s_i) = \{s_j / (s_i,s_j) \in A \text{ ou } (s_j,s_i) \in A\}$$
- Dans un graphe orienté, on distingue les sommets **successeurs** des sommets **prédécesseurs** :
$$succ(s_i) = \{s_j / (s_i,s_j) \in A\}$$
$$pred(s_i) = \{s_j / (s_j,s_i) \in A\}$$

1.1.2 Notion de degré d'un sommet

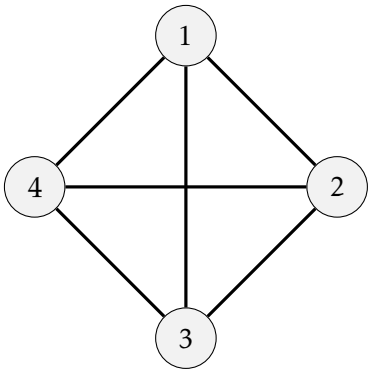
- Dans un graphe non orienté, le **degré** d'un sommet est le nombre d'arêtes incidentes à ce sommet (dans le cas d'un graphe simple, on aura $d^\circ(s_i) = |adj(s_i)|$).
- Dans un graphe orienté, le **demi-degré extérieur** d'un sommet s_i, noté $d^{\circ+}(s_i)$, est le nombre d'arcs partant de s_i (dans le cas d'un 1-graphe, on aura $d^{\circ+}(s_i) = |succ(s_i)|$).
De même, le **demi-degré intérieur** d'un sommet s_i, noté $d^{\circ-}(s_i)$, est le nombre d'arcs arrivant à s_i (dans le cas d'un 1-graphe, on aura $d^{\circ-}(s_i) = |pred(s_i)|$).

Activité 1.1.1

1. Dessiner un graphe non orienté complet à 4 sommets.
2. Quel est le degré des sommets de ce graphe ?
3. Combien d'arêtes possède-t-il ? Généralisez ces résultats à un graphe simple complet ayant n sommets.

Représentation du graphe :

Représentation de K_4



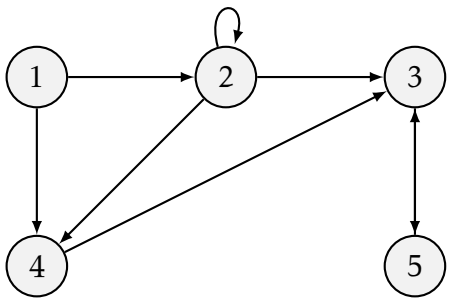
Activité 1.1.2

On considère le graphe orienté $G = (S,A)$ tel que :

$$S = \{1,2,3,4,5\}, \quad A = \{(1,2),(1,4),(2,2),(2,3),(2,4),(3,5),(4,3),(5,3)\}.$$

1. Représenter graphiquement ce graphe.
2. Donner le demi-degré extérieur de 2 et le demi-degré intérieur de 4.
3. Donner les sommets prédécesseurs de 4 et les sommets successeurs de 2.
4. Donner un graphe partiel et un sous-graphe de ce graphe.

Représentation du graphe :



Activité 1.1.3

Au cours d'une soirée, les convives se serrent les mains les uns les autres (jamais

plusieurs fois avec la même personne). Chacun se souvient du nombre de mains qu'il a serrées.

1. Montrer qu'il y a au moins deux personnes ayant serré le même nombre de mains.
2. Montrer que le nombre total de mains serrées est pair.
3. En déduire que le nombre de personnes ayant serré un nombre impair de mains est pair.

On construit le graphe non orienté $G = (S, A)$, où :

- S associe un sommet à chaque convive ;
- A associe une arête à chaque couple de convives qui se sont serré la main.

Le nombre de mains serrées par une personne correspond alors au **degré** du sommet correspondant dans le graphe.

1. Deux personnes ont serré le même nombre de mains. Montrer qu'il y a au moins deux personnes ayant serré le même nombre de mains revient à montrer qu'il existe au moins deux sommets ayant le même degré.
S'il y a n sommets, le degré d'un sommet est compris entre 0 (sommet isolé, c'est-à-dire la personne n'a serré aucune main) et $n-1$ (sommet relié à tous les autres, c'est-à-dire la personne a serré la main de tout le monde).
Pour que tous les sommets aient un degré différent, il faudrait qu'il y ait exactement :

$$\text{un sommet de degré } 0, \textit{un} \text{ de degré } 1, \ldots, \text{ et un de degré } n-1.$$

Mais, s'il existe un sommet de degré $n-1$, il ne peut y avoir de sommet de degré 0, car le sommet de degré $n-1$ serait relié à tous les autres. Cette contradiction montre qu'il existe nécessairement au moins deux sommets de même degré.

2. Le nombre total de mains serrées est pair. Montrer que le nombre total de mains serrées est pair revient à montrer que la somme des degrés de tous les sommets est paire. Chaque arête ajoute 1 au degré de deux sommets. Ainsi, chaque arête contribue pour 2 à la somme totale des degrés, d'où :

$$\sum_{s_i \in S} d(s_i) = 2|A|.$$

Cette somme est donc nécessairement paire.

3. Le nombre de personnes ayant serré un nombre impair de mains est pair. Montrer que le nombre de personnes ayant serré un nombre impair de mains est pair revient à montrer que le nombre de sommets de degré impair est pair.
On partitionne l'ensemble S des sommets en deux sous-ensembles :

$$S_{\text{pairs}} = \{s_i \in S \mid d(s_i) \text{ est pair}\}, \quad S_{\text{impairs}} = \{s_i \in S \mid d(s_i) \text{ est impair}\}.$$

On a alors :

$$\sum_{s_i \in S} d(s_i) = \sum_{s_j \in S_{\text{pairs}}} d(s_j) + \sum_{s_k \in S_{\text{impairs}}} d(s_k).$$

La somme totale $\sum_{s_i \in S} d(s_i)$ est paire (d'après la question 2), et la somme sur les degrés pairs $\sum_{s_j \in S_{\text{pairs}}} d(s_j)$ est aussi paire. Par conséquent, la somme $\sum_{s_k \in S_{\text{impairs}}} d(s_k)$ doit être paire.
Or, la somme d'un nombre impair d'entiers impairs est impaire. Donc, pour que cette somme soit paire, il faut que le nombre d'éléments dans S_{impairs} soit pair.
Ainsi, il y a un **nombre pair de sommets de degré impair**.

En résumé, pour tout graphe simple non orienté :

- il existe au moins deux sommets ayant le même degré ;
- la somme des degrés de tous les sommets est paire et égale à deux fois le nombre d'arêtes ;
- le nombre de sommets de degré impair est pair.

Définition 1.1.3

Une chaîne (ou un chemin) dans un graphe est une séquence ordonnée de sommets, où chaque paire consécutive de sommets est reliée par une arête.
Un chemin P de longueur k dans un graphe $G = (V, E)$ est une séquence de sommets $P : v_0, v_1, \cdots, v_k$ telle que :

- $v_i \in V \ \forall \ 0 \leq i \leq k$;
- $(v_i, v_{i+1}) \in E \ \forall \ 0 \leq i < k$ (les sommets successifs sont reliés par des arêtes).

Remarque 1.1.2

On distingue :

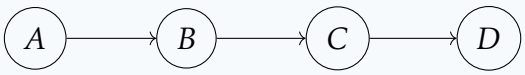
- **les chaînes simples :** Les sommets sont distincts, c'est-à-dire qu'aucun sommet n'est visité plus d'une fois (sauf éventuellement le point de départ si on revient dessus).
- **les chaînes fermées :** Le point de départ v_0 et le point d'arrivée v_k sont identiques.

Exemple 1.1.1

Une chaîne entre les sommets A,B,C,D :

$$P : A \to B \to C \to D$$

Chaîne simple
Une chaîne entre les sommets A,B,C,D :



Définition 1.1.4

Un cycle est une chaîne fermée où :

1. Le point de départ et d'arrivée sont identiques ($v_0 = v_k$).

2 Tous les autres sommets sont distincts (aucun sommet, à part v_0, n'est visité plus d'une fois).

3 La longueur d'un cycle est au moins égale à 3 (car une chaîne de longueur 2 fermée ne forme pas un cycle significatif).

Remarque 1.1.3

Un cycle C est une chaîne fermée $v_0, v_1, \cdots, v_k, v_0$ telle que :

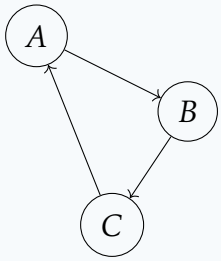
- $k \geq 3$;
- $(v_i, v_{i+1}) \in E \ \forall \ 0 \leq i < k$;
- $v_i \neq v_j \ \forall \ 0 \leq i, j < k$ si $i \neq j$.

Exemple 1.1.2

Un cycle entre les sommets A,B,C :

$$C : A \to B \to C \to A$$

Un cycle $A \to B \to C \to A$:



1.2 Représentation des graphes

Il existe deux façons classiques de représenter un graphe sur une machine :

- par une **matrice d'adjacence** ;
- ou par un **ensemble de listes d'adjacence**.

1.2.1 Matrice d'adjacence

Définition 1.2.1

Soit le graphe $G = (S,A)$. On suppose que les sommets de S sont numérotés de 1 à n, avec $n = |S|$.
La **matrice d'adjacence** de G est une matrice booléenne M de taille $n \times n$ telle que :

$$M[i][j] = \begin{cases} 1 & \text{si } (i,j) \in A, \\ 0 & \text{sinon.} \end{cases}$$

Si le graphe est **valué** (par exemple, si des distances sont associées aux arcs), on peut utiliser une **matrice d'entiers**, de telle sorte que :

$$M[i][j] = \begin{cases} \text{valeur de l'arc } (i,j) & \text{si } (i,j) \in A, \\ 0, \infty \text{ ou nu11} & \text{sinon.} \end{cases}$$

Dans le cas d'un **graphe non orienté**, la matrice d'adjacence est **symétrique** par rapport à sa diagonale principale. Ainsi, il suffit de mémoriser uniquement la **partie triangulaire supérieure** de la matrice.
Si le graphe est **non orienté**, alors la matrice d'adjacence est symétrique, c'est-à-dire $a_{ij} = a_{ji}$. Si le graphe est **orienté**, alors la matrice peut ne pas être symétrique.

Exemple 1.2.1

Considérons le graphe $G = (V, E)$ avec les sommets $V = \{v_1, v_2, v_3, v_4\}$ et les arêtes $E = \{(v_1, v_2), (v_2, v_3), (v_3, v_4)\}$. La matrice d'adjacence A du graphe est :

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Ici :

- $a_{12} = 1$ car (v_1, v_2) est une arête,
- $a_{23} = 1$ car (v_2, v_3) est une arête,
- $a_{34} = 1$ car (v_3, v_4) est une arête.

Tous les autres a_{ij} sont égaux à 0 car il n'y a pas d'arêtes entre ces sommets.

Proposition 1.2.1

- **Symétrie** : Pour un graphe non orienté, la matrice d'adjacence est symétrique, c'est-à-dire $a_{ij} = a_{ji}$.
- **Degré des sommets** : Le degré d'un sommet v_i dans un graphe peut être trouvé en calculant la somme des éléments de la ligne (ou de la colonne, dans le cas d'un graphe non orienté) correspondant à v_i.
  Pour un graphe orienté, on a :
  — Degré extérieur pour v_i : $d^+(v_i) = \sum_j a_{ij}$,
  — Degré intérieur pour v_i : $d^-(v_i) = \sum_j a_{ji}$.

Théorème 1.2.1

On considère un graphe d'ordre n et on note M sa matrice d'adjacence.
Le nombre de chemins de longueur p reliant deux sommets i et j est donné par le terme de la i-ième ligne et de la j-ième colonne de la matrice M^p.

Applications

La matrice d'adjacence est utilisée pour :

- **Trouver les chemins** entre les sommets, en calculant des puissances successives de la matrice. Par exemple, A^2 indique l'existence de chemins de longueur 2 entre les sommets.
- **Calculer le nombre de triangles** dans un graphe, en utilisant le produit matriciel de A avec lui-même.

La matrice d'adjacence est un outil fondamental pour l'étude algorithmique des graphes. Elle permet de représenter les graphes de manière compacte et d'effectuer des calculs efficaces sur les structures graphiques, comme la recherche de chemins, de cycles, ou de connexions.

1.2.2 Liste d'adjacence

Soit le graphe $G = (S,A)$. On suppose que les sommets de S sont numérotés de 1 à n, avec $n = |S|$. La représentation par **listes d'adjacence** de G consiste en un tableau T de n listes, une pour chaque sommet de S.

Pour chaque sommet $s_i \in S$, la liste d'adjacence $T[s_i]$ est une liste chaînée de tous les sommets s_j tels qu'il existe un arc ou une arête $(s_i,s_j) \in A$. Autrement dit, $T[s_i]$ contient la liste de tous les sommets *successeurs* de s_i. Les sommets de chaque liste d'adjacence sont généralement chaînés selon un ordre arbitraire.

Si le graphe est **valué** (par exemple, si les arêtes représentent des distances), on peut stocker dans les listes d'adjacence, en plus du numéro de sommet, la *valuation* de l'arête.

Dans le cas de **graphes non orientés**, pour chaque arête (s_i,s_j), on aura s_j qui appartiendra à la liste chaînée de $T[s_i]$, et aussi s_i qui appartiendra à la liste chaînée de $T[s_j]$.

Exemple 1.2.2

Prenons le graphe $G = (V,E)$ avec les sommets $V = \{v_1,v_2,v_3,v_4\}$ et les arêtes $E = \{(v_1,v_2),(v_2,v_3),(v_3,v_4)\}$. La liste d'adjacence correspondante est :

$$L = \begin{cases} L(v_1) = \{v_2\}, \\ L(v_2) = \{v_1,v_3\}, \\ L(v_3) = \{v_2,v_4\}, \\ L(v_4) = \{v_3\}. \end{cases}$$

Proposition 1.2.2

- **Représentation compacte** : La liste d'adjacence est plus compacte que la matrice d'adjacence, car elle ne nécessite de stocker que les arêtes existantes, sans redondance.
- **Efficacité pour les graphes peu denses** : Elle est particulièrement efficace pour les graphes avec un nombre d'arêtes relativement faible par rapport au nombre de sommets.
- **Accès aux voisins** : L'accès aux voisins d'un sommet v_i se fait en parcourant la liste $L(v_i)$, ce qui peut être plus rapide et plus économique en mémoire que

dans la matrice d'adjacence.

- **Graphe orienté** : Dans le cas des graphes orientés, la liste d'adjacence représente les arêtes sortantes. Par conséquent, la liste $L(v_i)$ contient les sommets v_j tels que $(v_i, v_j) \in E$, mais pas les arcs allant dans le sens inverse.

Applications

La liste d'adjacence est utilisée dans les algorithmes de parcours de graphe, comme l'algorithme de **parcours en largeur** (BFS) ou le **parcours en profondeur** (DFS), car elle permet un accès rapide aux voisins d'un sommet. Elle est également utilisée dans les algorithmes de recherche de chemins, comme l'algorithme de Dijkstra ou l'algorithme de Bellman-Ford, où il est nécessaire de parcourir rapidement les voisins de chaque sommet. La liste d'adjacence est une représentation très efficace pour les graphes peu denses, et elle permet de réaliser des opérations sur les graphes avec un faible coût en mémoire. Sa simplicité et sa performance dans les algorithmes de recherche et de parcours en font un outil fondamental dans l'analyse des graphes.

1.2.3 Degré d'un sommet

Définition 1.2.2

On appelle **degré** d'un sommet v, et on le note $d(v)$, le nombre d'arêtes incidentes à ce sommet.

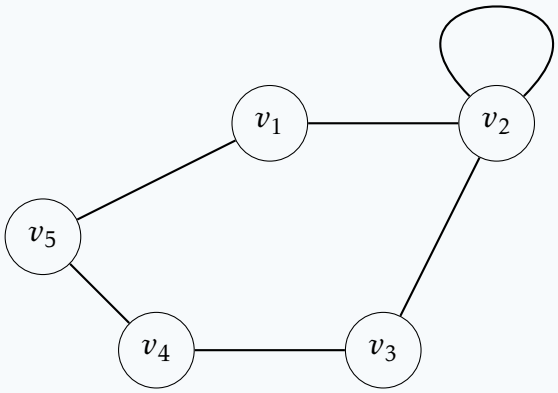
Remarque 1.2.1

Une boucle sur un sommet est comptée deux fois dans le degré de ce sommet.
Dans un graphe simple, le degré d'un sommet peut aussi être défini comme le nombre de ses voisins, c'est-à-dire la taille de son voisinage.

Exemple 1.2.3

Dans le multigraphe ci-dessous, les degrés des sommets sont les suivants :

- $d(v_1) = \ldots$
- $d(v_2) = \ldots$
- $d(v_3) = \ldots$



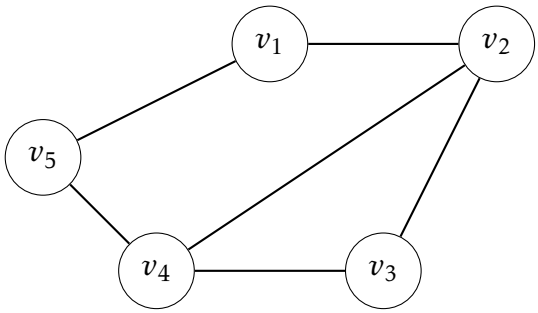
Théorème 1.2.2

La somme des degrés des sommets d'un graphe est égale à deux fois le nombre d'arêtes.

1.2.4 Degré d'un graphe

Définition 1.2.3

Le degré d'un graphe est le degré maximum de tous ses sommets. Autrement dit, le degré d'un graphe est le plus grand des degrés des sommets qui le composent. Dans l'exemple ci-dessous, le degré du graphe est 3, à cause du sommet v_2.



Remarque 1.2.2

Un graphe dont tous les sommets ont le même degré est dit **régulier**. Si le degré commun est k, alors on dit que le graphe est k-**régulier**.

Chapitre 2

Propriétés fondamentales des graphes

Sommaire

2.1 Connexité : composantes connexes, graphes fortement connexes 13
2.2 Graphes eulériens 15
2.3 Graphe hamiltonien 16
2.4 Arbre et arborescence 17
2.5 Coloriage de graphes, cliques et stables 19

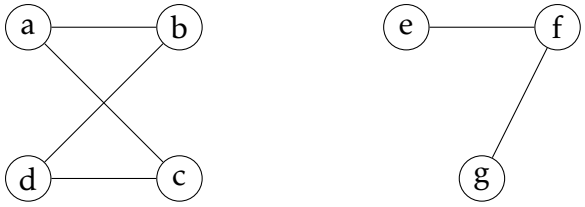
2.1 Connexité : composantes connexes, graphes fortement connexes

2.1.1 Cas des graphes non orientés

Définition 2.1.1

Un graphe non orienté est **connexe** si chaque sommet est accessible à partir de n'importe quel autre. Autrement dit, si pour tout couple de sommets distincts $(s_i,s_j) \in S^2$, il existe une chaîne entre s_i et s_j.

Par exemple, le graphe non orienté suivant n'est pas connexe :



car il n'existe pas de chaîne entre les sommets a et e. En revanche, le sous-graphe défini par les sommets $\{a,b,c,d\}$ est connexe.

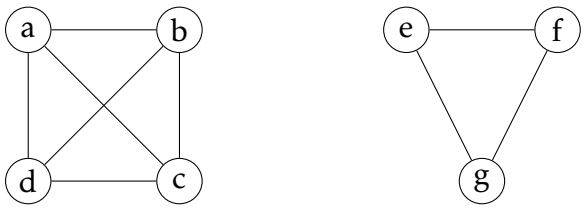
Définition 2.1.2

Une **composante connexe** d'un graphe non orienté G est un sous-graphe G' de G qui est connexe et maximal (c'est-à-dire qu'aucun autre sous-graphe connexe de G ne contient G').

Par exemple, le graphe précédent est composé de deux composantes connexes : la première est le sous-graphe défini par les sommets $\{a,b,c,d\}$ et la seconde est le sous-graphe défini par les sommets $\{e,f,g\}$.

Notons que si l'on calcule la fermeture transitive d'un graphe connexe, on obtient un graphe complet. De même, si l'on calcule la fermeture transitive d'un graphe comportant k

composantes connexes, on obtient un graphe contenant k sous-graphes complets (un pour chaque composante connexe). Par exemple, la fermeture transitive du graphe précédent est :



2.1.2 Cas des graphes orientés

On retrouve ces différentes notions de connexités dans les graphes orientés, en remplaçant naturellement la notion de chaîne par celle de chemin : on parle de **graphe fortement connexe** au lieu de connexe, de **composante fortement connexe** au lieu de composante connexe.

Plus précisément, un graphe orienté est **fortement connexe** si chaque sommet est accessible à partir de n'importe quel autre. Autrement dit, si pour tout couple de sommets distincts $(s_i,s_j) \in S^2$, il existe un chemin de s_i vers s_j, et un chemin de s_j vers s_i. Par exemple, le graphe de gauche ci-dessous est fortement connexe, tandis que celui de droite ne l'est pas :



Définition 2.1.3

Une **composante fortement connexe** d'un graphe orienté G est un sous-graphe G' de G qui est fortement connexe et maximal (c'est-à-dire qu'aucun autre sous-graphe fortement connexe de G ne contient G').

Par exemple, le graphe orienté suivant :



contient 2 composantes fortement connexes : la première est le sous-graphe défini par les sommets $\{a,b,c,d\}$ et la seconde est le sous-graphe défini par les sommets $\{e,f,g\}$.

Comme pour les graphes non orientés, une façon (naïve) de déterminer si un graphe orienté est fortement connexe consiste à calculer sa fermeture transitive : si la fermeture transitive du graphe est le graphe complet, alors il est fortement connexe. Notons qu'il existe un algorithme bien plus efficace pour déterminer les composantes fortement connexes d'un graphe non orienté.

2.2 Graphes eulériens

Définition 2.2.1

Dans un graphe non orienté, une **chaîne eulérienne** est une chaîne qui emprunte une et une seule fois chaque arête du graphe. De même, un **cycle eulérien** est un cycle qui emprunte une et une seule fois chaque arête du graphe. Enfin, un graphe comportant une chaîne ou un cycle eulérien est appelé **graphe eulérien**.

Théorème 2.2.1

Un graphe (simple ou multiple) connexe admet un cycle eulérien si et seulement s'il n'a pas de sommet de degré impair.

Preuve. En effet, supposons qu'un graphe $G = (V,E)$ admette un cycle eulérien. Chaque fois que ce cycle passe par un sommet, il contribue pour 2 dans le degré de ce sommet. Par conséquent, chaque sommet doit avoir un degré pair.
Inversement, soit G un graphe connexe dont tous les sommets sont de degré pair. On montre, par induction sur le nombre d'arêtes du graphe, que ce graphe admet un cycle eulérien. Partant d'un sommet arbitraire a, on peut former de proche en proche un cycle élémentaire retournant sur a. En effet, chaque fois qu'on emprunte une arête, on la retire du graphe. Quand on aboutit en un sommet $x \neq a$, on peut toujours prolonger la chaîne car x est de degré pair. Donc, la chaîne rencontrera nécessairement le sommet initial a. Soit μ le cycle formé à ce moment-là. En vertu de l'hypothèse d'induction, chaque composante connexe de sous-graphe $G_{X-\mu}$ admet un cycle eulérien, et avec le cycle μ ces cycles forment un cycle eulérien de G. □

Théorème 2.2.2

Un graphe (simple ou multiple) connexe admet une chaîne eulérienne entre deux sommets u et v si et seulement si le degré de u et le degré de v sont impairs, et les degrés de tous les autres sommets du graphe sont pairs.

On retrouve ces différentes notions sur les graphes orientés : un **chemin eulérien** est un chemin qui emprunte une et une seule fois chaque arc du graphe. De même, un **circuit eulérien** est un circuit qui emprunte une et une seule fois chaque arc du graphe. L'existence d'un circuit eulérien dépend alors des demi-degrés extérieurs et intérieurs des différents sommets : pour chaque sommet, il doit y avoir autant d'arcs qui y arrivent que d'arcs qui en partent.

Théorème 2.2.3

Un multigraphe orienté fortement connexe admet un circuit eulérien si et seulement si $d^+(s_i) = d^-(s_i)$ pour tout sommet $s_i \in S$.

Théorème 2.2.4

Un multigraphe orienté connexe admet un chemin eulérien de u vers v si et seulement

si :

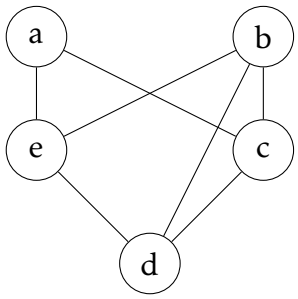
$$\begin{cases} d^+(u) = d^-(u) + 1, \\ d^+(v) = d^-(v) - 1, \\ d^+(s_i) = d^-(s_i), \quad \text{pour tout autre sommet } s_i \in S - \{u,v\}. \end{cases}$$

2.3 Graphe hamiltonien

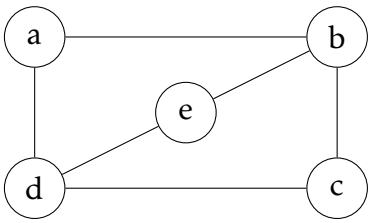
Définition 2.3.1

Dans un graphe simple non orienté comportant n sommets, une **chaîne hamiltonienne** est une chaîne élémentaire de longueur $n-1$. Autrement dit, une chaîne hamiltonienne passe une et une seule fois par chacun des n sommets du graphe. On appelle **cycle hamiltonien** un cycle élémentaire de longueur n. Un graphe possédant un cycle ou une chaîne hamiltonien sera dit **graphe hamiltonien**.

Par exemple, le graphe suivant possède un cycle hamiltonien (a,e,b,d,c,a) :



En revanche, le graphe suivant ne possède pas de cycle hamiltonien, mais possède une chaîne hamiltonienne (a,b,e,d,c) :



On ne connaît aucune condition nécessaire et suffisante d'existence de cycles (chaînes, circuits ou chemins) hamiltoniens, valable pour tous les graphes. On peut juste donner des conditions suffisantes, portant en particulier sur les degrés d'un graphe simple.

Remarque 2.3.1

De nombreux problèmes en recherche opérationnelle consistent à chercher un chemin ou un cycle hamiltonien dans un graphe. Le plus connu est probablement le *problème du voyageur de commerce*, qui doit trouver un itinéraire "optimal" passant par chaque ville de son réseau commercial. On considère dans ce cas le graphe non orienté valué représentant une carte routière : les sommets correspondent aux villes, les arêtes aux routes, et les valuations aux distances. Il s'agit alors de trouver un cycle hamiltonien de valuation minimale. Ce problème est un des premiers à avoir été montré comme

étant **NP-complet** (ce qui implique que l'on ne connaît aucun algorithme "efficace" pour le résoudre en général).

2.4 Arbre et arborescence

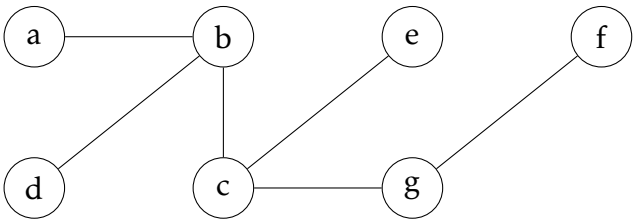
Les arbres et les arborescences sont des graphes particuliers très souvent utilisés en informatique pour représenter des données.

Définition 2.4.1

Etant donné un graphe non orienté comportant n sommets, les propriétés suivantes sont équivalentes pour caractériser **un arbre** :

1. G est connexe et sans cycle,
2. G est sans cycle et possède n 1 arêtes,
3. G est connexe et admet n 1 arêtes,
4. G est sans cycle, et en ajoutant une arête, on crée un et un seul cycle élémentaire,
5. G est connexe, et en supprimant une arête quelconque, il n'est plus connexe,
6. Il existe une chaine et une seule entre 2 sommets quelconques de G.

Par exemple, le graphe suivant est un arbre :

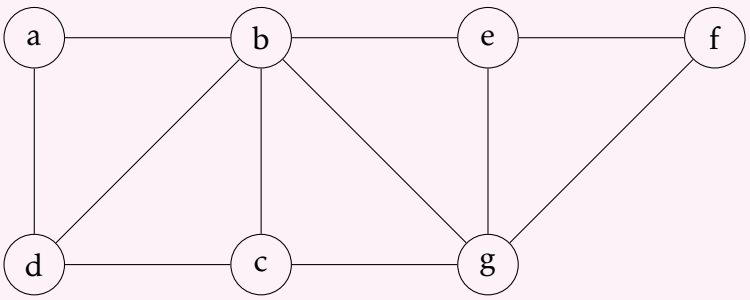


Définition 2.4.2

On appelle **forêt** un graphe dont chaque composante connexe est un arbre

Activité 2.4.1

On considère le graphe non orienté suivant :



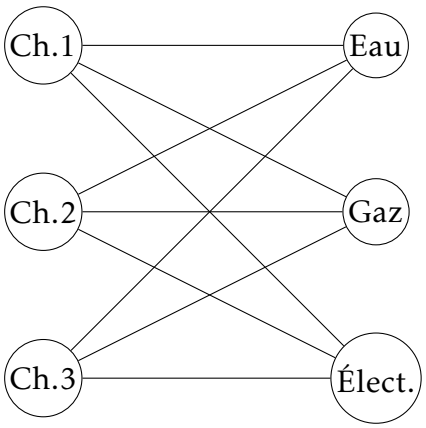
1. Combien faut-il enlever d'arêtes à ce graphe pour le transformer en arbre ?
2. Donnez un graphe partiel de ce graphe qui soit un arbre.

2.4.1 Planarité et coloriage de graphes

Exemple : problème des trois châtelains

Trois châtelains voisins sont ennemis entre eux. Pour se mettre au goût du jour, ils ont décidé d'installer l'eau, le gaz et l'électricité dans leurs châteaux respectifs. Naturellement, ils ne veulent pas que leurs fils et leurs canalisations se croisent !
Il s'agit donc de trouver le moyen de relier chacun des châtelains aux compagnies d'eau, de gaz et d'électricité sans provoquer de disputes entre eux.
On peut modéliser ce problème par un graphe ayant les six sommets suivants :

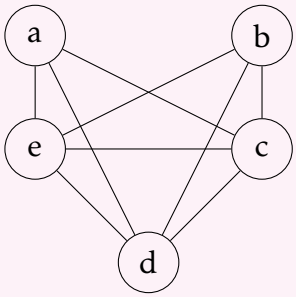
- les trois châteaux : *Château 1*, *Château 2*, *Château 3* ;
- les trois compagnies : *eau*, *gaz*, *électricité*.



Il s'agit alors de dessiner pour chaque château trois arêtes le reliant à l'eau, au gaz et à l'électricité, sans que deux arêtes ne se croisent.
Ce problème n'a en fait pas de solution (même en changeant la disposition des sommets). On dira que le graphe correspondant, appelé $K_{3,3}$, **n'est pas planaire.**
De façon plus formelle, on appelle graphe planaire tout graphe non orienté pouvant être dessiné sur un plan de telle sorte que les sommets soient des points distincts, et que les arêtes ne se rencontrent pas en dehors de leurs extrémités (les arêtes pouvant être représentées par des courbes).
Si on considère par exemple une carte de géographie, et si on associe un sommet à chaque pays et une arête entre deux sommets si les pays correspondants ont une frontière commune (non réduite à un seul point), alors on obtient un graphe planaire.

Activité 2.4.2

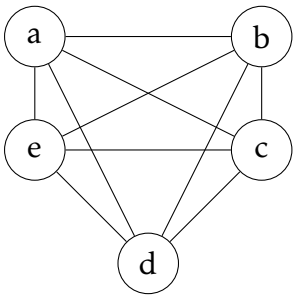
1. On appelle K_i le graphe complet comportant i sommets. Parmi K_2 , K_3 , K_4 et K_5, lesquels sont planaires ?
2. Montrer que le graphe suivant est planaire :

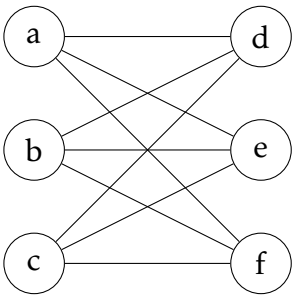


Remarque 2.4.1

L'adjectif "planaire" qualifie un graphe, indépendemment de sa représentation gra phique choisie. Autrement dit, un même graphe peut être dessiné de différentes façons (comme dans l'exercice précédent). Certaines de ces représentations peuvent ne pas être planaires, alors que d'autres peuvent l'être. Il suffit, pour un graphe donné, de trouver une représentation planaire pour en conclure que le graphe planaire.

Caractérisation des graphes planaires : Les deux graphes non planaires les plus simples sont K_5 et $K_{3,3}$:





2.5 Coloriage de graphes, cliques et stables

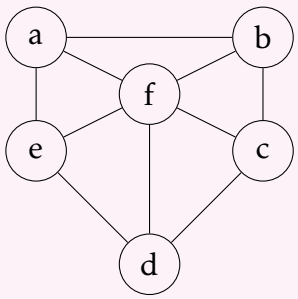
Le problème du **coloriage de graphes**, pour un graphe non orienté G, consiste à attribuer une couleur à chaque sommet, de telle sorte qu'une même couleur ne soit pas attribuée à deux sommets adjacents (reliés par une arête).
Lorsque le graphe est planaire, ce problème peut s'exprimer par rapport au **graphe dual**, en coloriant les faces, et non les sommets, et en imposant que deux faces voisines soient coloriées de couleurs différentes (il s'agit du *problème de coloriage d'une carte*).
Le nombre minimum de couleurs nécessaires pour colorier un graphe G est appelé le **nombre chromatique** de G, et noté $\chi(G)$.
Remarque : le problème du coloriage d'un graphe avec un nombre limité de couleurs est un **problème combinatoire** (on dit qu'il s'agit d'un problème **NP-complet**). Cela signifie que tout algorithme résolvant ce problème de façon exacte pourra prendre un temps *exponentiel* par rapport au nombre de sommets du graphe (de l'ordre de 2^n pour n sommets).
Le problème de déterminer le nombre chromatique d'un graphe est également exponentiel, et est en fait encore plus difficile.
L'**algorithme de Brélaz** (également appelé **DSATUR**) est un *algorithme glouton* qui permet de calculer une borne maximale de $\chi(G)$. Cet algorithme a une complexité polynomiale et procède de façon itérative.

Plus précisément, tant qu'il existe un sommet non colorié :

- on choisit le sommet non colorié ayant le plus grand nombre de voisins déjà coloriés avec des couleurs différentes ;
- les ex aequo sont départagés en choisissant le sommet de plus fort degré ;
- le sommet choisi est alors colorié par la plus petite couleur possible (en partant du principe que les couleurs sont triées selon un ordre donné) ;
- si toutes les couleurs existantes sont utilisées par au moins un voisin du sommet à colorier, alors une nouvelle couleur est ajoutée à l'ensemble des couleurs disponibles.

Activité 2.5.1

Déterminez le nombre chromatique du graphe suivant :



Conjecture des 4 couleurs : Le nombre chromatique d'un graphe planaire est inférieur ou égal à 4.
Cette conjecture a passionné de nombreux mathématiciens, pendant plus d'un siècle. Elle a été démontrée en 1976.
Une application de cette conjecture est que l'on peut colorier les pays d'une carte géographique avec seulement 4 couleurs de telle sorte que deux pays voisins soient coloriés avec des couleurs différentes.

2.5.1 Cliques et ensembles stables

Définition 2.5.1

Étant donné un graphe non orienté $G = (S,A)$, une **clique** est un sous-ensemble de sommets $S' \subseteq S$ qui sont tous connectés deux à deux par des arêtes, de sorte que :

$$\forall (i,j) \in S' \times S',\ i \neq j \Rightarrow (i,j) \in A$$

Dit autrement, une clique est un *sous-graphe complet*.

Trouver une clique d'ordre k dans un graphe est également un **problème NP-complet**, ce qui implique là encore que tout algorithme résolvant ce problème de façon exacte aura une complexité exponentielle. Le problème de trouver la plus grande clique d'un graphe est appelé **problème de la clique maximum**, et est encore plus difficile...

Activité 2.5.2

Montrer que pour tout graphe G, l'ordre de la clique maximum est inférieur ou égal à $\chi(G)$.

Définition 2.5.2

Étant donné un graphe non orienté $G = (S,A)$, un **stable** est un sous-ensemble de sommets $S' \subseteq S$ non connectés par des arêtes, de sorte que :

$$\forall (i,j) \in S' \times S',\ i \neq j \Rightarrow (i,j) \notin A$$

Le problème de rechercher un stable d'ordre k dans un graphe $G = (S,A)$ est équivalent au problème de rechercher une clique d'ordre k dans le **graphe inverse** $G' = (S,A')$ tel que A' contient une arête entre i et j si et seulement si A ne contient pas d'arête entre ces deux sommets.

Chapitre 3

Les algorithmes sur les graphes

Sommaire

3.1 Parcours de graphes : parcours en largeur (BFS) et en profondeur (DFS) 22
3.2 Algorithmes de recherche de chemins : Dijkstra, Bellman-Ford 25
3.3 Algorithmes pour arbres couvrants : Kruskal, Prim 30
3.4 Flots dans les réseaux : Algorithme de Ford-Fulkerson 33

3.1 Parcours de graphes : parcours en largeur (BFS) et en profondeur (DFS)

Les algorithmes de parcours constituent la brique fondamentale de l'algorithmique des graphes. Parcourir un graphe $G = (V,E)$, où V désigne l'ensemble des sommets ($|V| = n$) et E l'ensemble des arêtes ou arcs ($|E| = m$), consiste à explorer de façon système tous les sommets accessibles à partir d'un sommet source donné $s \in V$.

3.1.1 Parcours en largeur (BFS – Breadth-First Search)

3.1.1.1 A. Principe et fondements théoriques

Le parcours en largeur explore le graphe par « couches » ou niveaux de distance croissante par rapport au sommet source s :

1. On visite d'abord le sommet source s (niveau 0).
2. Puis tous les voisins directs de s (niveau 1, situés à une distance de 1 arête).
3. Puis les voisins non visités des sommets du niveau 1 (niveau 2, situés à une distance de 2 arêtes), et ainsi de suite.

Pour garantir cet ordre d'exploration (premier découvert, premier traité), le BFS s'appuie obligatoirement sur une structure de données de type **FILE (Queue – FIFO : First In, First Out)**.

3.1.1.2 B. Spécification de l'algorithme

On conserve trois attributs pour chaque sommet $u \in V$:

- couleur$[u]$: `BLANC` (non découvert), `GRIS` (découvert, en attente dans la file), `NOIR` (traité).
- $d[u]$: la distance minimale (nombre d'arêtes) entre s et u.
- $\pi[u]$: le prédecesseur de u dans l'arbre de parcours BFS.

Définition 3.1.1 Algorithme du Parcours en Largeur (BFS)
Entrée : Un graphe $G = (V,E)$ et un sommet source $s \in V$

Sortie : Tableaux des distances d et des pères π

1. **Pour chaque** sommet $u \in V \setminus \{s\}$ **faire**
   - couleur$[u] \leftarrow$ BLANC, $d[u] \leftarrow +\infty$, $\pi[u] \leftarrow$ NIL
2. couleur$[s] \leftarrow$ GRIS, $d[s] \leftarrow 0$, $\pi[s] \leftarrow$ NIL
3. $F \leftarrow$ CréerFileVide()
4. Enfiler(F,s)
5. **Tant que** $F \neq \emptyset$ **faire**
   - $u \leftarrow$ Défiler(F)
   - **Pour chaque** voisin $v \in$ Adjacents$[u]$ **faire**
     - **Si** couleur$[v]$ == BLANC **alors**
       - couleur$[v] \leftarrow$ GRIS
       - $d[v] \leftarrow d[u] + 1$
       - $\pi[v] \leftarrow u$
       - Enfiler(F,v)
   - couleur$[u] \leftarrow$ NOIR

3.1.1.3 C. Propriétés et Complexité

- **Complexité temporelle :** $\mathcal{O}(|V| + |E|)$ avec une représentation par listes d'adjacence.
- **Complexité spatiale :** $\mathcal{O}(|V|)$ pour la gestion de la file F et des vecteurs d'état.
- **Propriété du plus court chemin :** Dans un graphe non pondéré, pour tout sommet $u \in V$, $d[u]$ calculé par le BFS est exactement égal à la distance géodésique $\delta(s,u)$.

3.1.2 Parcours en profondeur (DFS – Depth-First Search)

3.1.2.1 A. Principe et fondements théoriques

Le parcours en profondeur explore un chemin aussi loin que possible le long de chaque branche avant de revenir en arrière (*backtracking*). Dès qu'un voisin non découvert est rencontré, l'exploration bifurque immédiatement vers ce sommet. Le DFS s'appuie sur une structure de **PILE (Stack – LIFO : Last In, First Out)**, implémentée naturellement par la pile d'appels récursifs.

3.1.2.2 B. Spécification de l'algorithme

En plus des structures standards, le DFS associe deux estampilles temporelles à chaque sommet u :

- $d_time[u]$: date de première découverte de u (passage à GRIS).
- $f_time[u]$: date de fin d'exploration de u et de toute sa descendance (passage à NOIR).

Définition 3.1.2 Algorithme du Parcours en Profondeur (DFS)
Entrée : Un graphe $G = (V,E)$

Sortie : Estampilles temporelles d_time, f_time et arborescence des pères π

Procédure Principale DFS(G) :

1. **Pour chaque** sommet $u \in V$ **faire**
   - couleur$[u] \leftarrow$ `BLANC`, $\pi[u] \leftarrow$ NIL
2. temps $\leftarrow 0$
3. **Pour chaque** sommet $u \in V$ **faire**
   - **Si** couleur$[u] ==$ `BLANC` **alors** DFS-Visiter(G, u)

Procédure Auxiliaire DFS-Visiter(G, u) :

a. temps $\leftarrow$ temps $+ 1$, $d_time[u] \leftarrow$ temps, couleur$[u] \leftarrow$ `GRIS`

b. **Pour chaque** voisin $v \in$ Adjacents$[u]$ **faire**
   - **Si** couleur$[v] ==$ `BLANC` **alors**
     - $\pi[v] \leftarrow u$
     - DFS-Visiter(G, v)

c. couleur$[u] \leftarrow$ `NOIR`

d. temps $\leftarrow$ temps $+ 1$, $f_time[u] \leftarrow$ temps

3.1.2.3 C. Classification des arêtes

Lors d'un DFS, les arêtes du graphe se divisent en 4 catégories :

1. **Arêtes d'arbre :** Arêtes appartenant à l'arbre de recouvrement du parcours.
2. **Arêtes arrières (*Back edges*) :** Relient un sommet à un de ses ancêtres (présence d'un cycle).
3. **Arêtes avant (*Forward edges*) :** Relient un sommet à un descendant non direct.
4. **Arêtes croisées (*Cross edges*) :** Relient deux sommets sans relation d'ascendance directe.

3.1.3 Illustration des parcours

Considérons le graphe non orienté $G = (V, E)$ représenté ci-dessous, constitué de 6 sommets $V = \{A, B, C, D, E, F\}$:
Convention d'implémentation : les voisins d'un sommet sont explorés par ***ordre alphabétique****.*

3.1.3.1 1. Déroulement détaillé du BFS (Sommet source : A)

3.1.3.2 2. Déroulement détaillé du DFS (Sommet source : A)

Notation des dates : `[d_time / f_time]` sur chaque sommet.

- **Ordre de découverte (pré-ordre) :** $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F$
- **Ordre de fermeture (post-ordre) :** $C \rightarrow F \rightarrow E \rightarrow D \rightarrow B \rightarrow A$

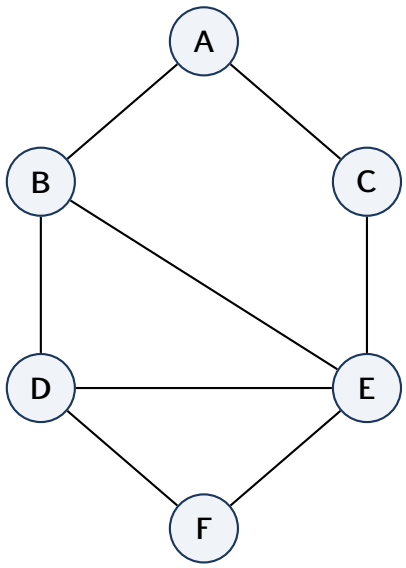


FIGURE 3.1.1 – Graphe exemple non orienté G.

Étape	Sommet extrait	Voisins examinés	File F	Découverts
0 (Init)	–	–	$[A]$	$\{A\}$
1	**A**	B (BLANC), C (BLANC)	$[B,C]$	$\{A,B,C\}$
2	**B**	A (NOIR), D (BLANC), E (BLANC)	$[C,D,E]$	$\{A,\ldots,E\}$
3	**C**	A (NOIR), E (GRIS)	$[D,E]$	$\{A,\ldots,E\}$
4	**D**	B (NOIR), E (GRIS), F (BLANC)	$[E,F]$	$\{A,\ldots,F\}$
5	**E**	B,C,D,F (tous vus)	$[F]$	$\{A,\ldots,F\}$
6	**F**	D,E (tous vus)	$[\,]$	$\{A,\ldots,F\}$

TABLE 3.1 – Trace d'exécution du BFS issu du sommet A.

3.2 Algorithmes de recherche de chemins : Dijkstra, Bellman-Ford

La recherche du plus court chemin dans un graphe valué (ou pondéré) consiste à déterminer une chaîne ou un chemin reliant deux sommets de telle sorte que la somme des poids des arêtes ou arcs qui le composent soit minimale.
Soit $G = (V,E)$ un graphe orienté ou non orienté, et $w : E \to \mathbb{R}$ une fonction de pondération. La longueur d'un chemin $p = \langle v_0,v_1,\ldots,v_k\rangle$ est définie par $w(p) = \sum_{i=1}^{k} w(v_{i-1},v_i)$. La distance le plus courte de u à v est notée $\delta(u,v)$.

3.2.1 Algorithme de Dijkstra

3.2.1.1 A. Principe et conditions d'application

L'algorithme de Dijkstra résout le problème des plus courts chemins à source unique dans un graphe orienté ou non, sous la condition stricte que **tous les poids soient non négatifs** ($\forall (u,v) \in E, w(u,v) \geq 0$).

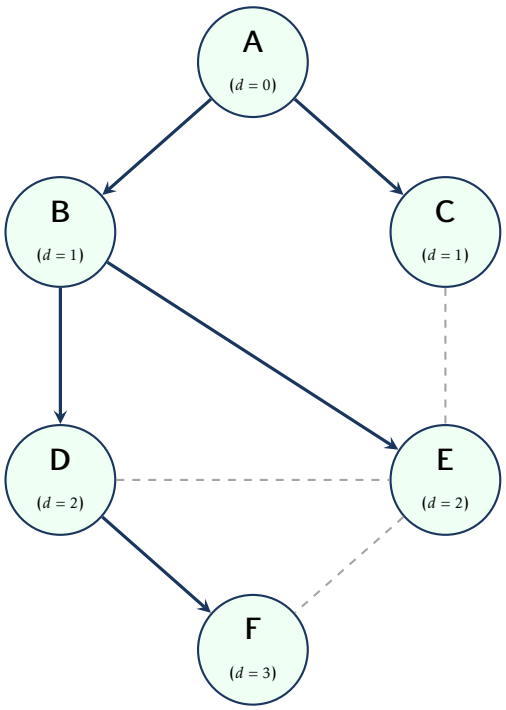


FIGURE 3.1.2 – Arbre BFS des plus courts chemins depuis A.

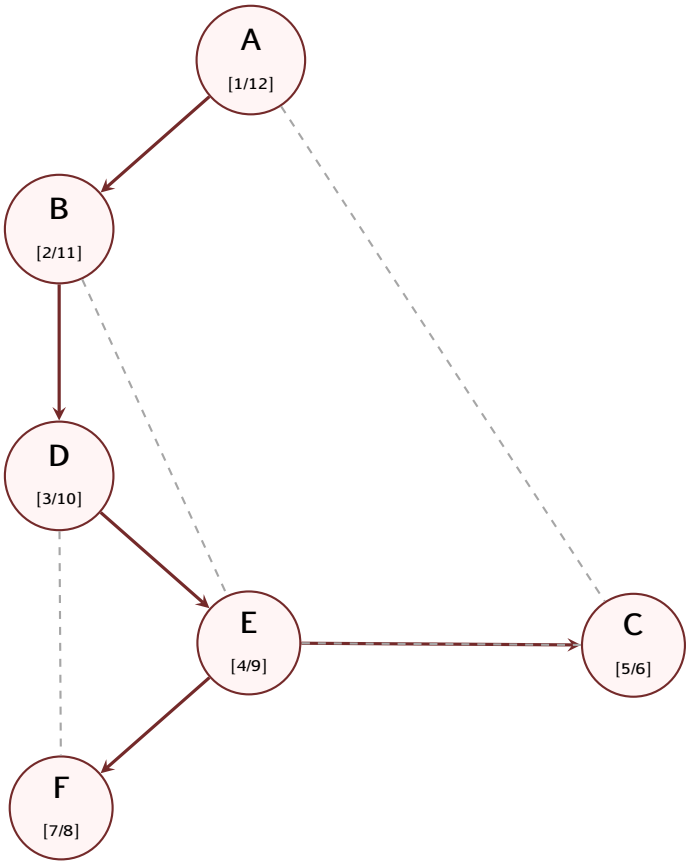


FIGURE 3.1.3 – Arbre d'exploration et estampilles temporelles DFS.

Il repose sur une approche **gloutonne** : on maintient un ensemble S de sommets dont les distances minimales depuis la source s sont définitivement connues. À chaque étape, on extrait du groupe des sommets restants ($V \setminus S$) le sommet u possédant la plus petite

estimation de distance $d[u]$, puis on procède au relâchement (*relaxation*) de toutes ses arêtes sortantes.

3.2.1.2 B. Spécification de l'algorithme

Définition 3.2.1 Procédure de Relâchement (*Relaxation*)

Le relâchement d'un arc (u,v) de poids $w(u,v)$ consiste à tester si le passage par u améliore la distance actuellement connue pour aller à v :

$$\textbf{Si } d[v] > d[u] + w(u,v) \textbf{ Alors } d[v] \leftarrow d[u] + w(u,v) \quad \text{et} \quad \pi[v] \leftarrow u$$

Définition 3.2.2 Algorithme de Dijkstra

Entrée : Graphe $G = (V,E)$, matrice/fonction des poids $w \geq 0$, sommet source $s \in V$
Sortie : Tableaux des distances minimales d et des pères π

1. **Pour chaque** sommet $v \in V$ **faire**
   - $d[v] \leftarrow +\infty, \quad \pi[v] \leftarrow \text{NIL}$
2. $d[s] \leftarrow 0$
3. $S \leftarrow \emptyset$ (ensemble des sommets définitivement traités)
4. $Q \leftarrow V$ (file de priorité contenant tous les sommets de V)
5. **Tant que** $Q \neq \emptyset$ **faire**
   - $u \leftarrow$ Extraire-Min(Q) (sommet $u \in Q$ minimisant $d[u]$)
   - $S \leftarrow S \cup \{u\}$
   - **Pour chaque** voisin $v \in$ Adjacents$[u]$ **faire**
     - **Si** $d[v] > d[u] + w(u,v)$ **alors**
       - $d[v] \leftarrow d[u] + w(u,v)$
       - $\pi[v] \leftarrow u$
       - Diminuer-Clé$(Q,v,d[v])$

3.2.1.3 C. Complexité

- Avec un tableau ordinaire pour Q : $\mathcal{O}(|V|^2)$.
- Avec un tas binaire (*Binary Heap*) : $\mathcal{O}((|V|+|E|)\log|V|)$.
- Avec un tas de Fibonacci : $\mathcal{O}(|E|+|V|\log|V|)$.

3.2.2 Algorithme de Bellman-Ford

3.2.2.1 A. Principe et détection de cycles négatifs

L'algorithme de Bellman-Ford résout le problème des plus courts chemins à source unique dans le cas général où **les arcs peuvent avoir des poids négatifs**.
Contrairement à Dijkstra, il utilise la **programmation dynamique** et effectue $|V|-1$ itérations de relâchement sur **la totalité des arcs** du graphe. Un $|V|$-ième relâchement

permet de détecter la présence d'un **cycle de poids strictement négatif** accessible depuis la source (ce qui rendrait la distance $-\infty$).

3.2.2.2 B. Spécification de l'algorithme

Définition 3.2.3 Algorithme de Bellman-Ford

Entrée : Graphe $G = (V,E)$, fonction des poids w, sommet source $s \in V$
Sortie : Booléen (`VRAI` si pas de cycle négatif, `FAUX` sinon), tableaux d et π

1. **Initialisation :**
   - **Pour chaque** $v \in V$ **faire** $d[v] \leftarrow +\infty$, $\pi[v] \leftarrow$ NIL
   - $d[s] \leftarrow 0$
2. **Relâchements répétitifs :**
   - **Pour** $i \leftarrow 1$ **à** $|V|-1$ **faire**
     - **Pour chaque** arc $(u,v) \in E$ **faire**
       - ▷ **Si** $d[v] > d[u] + w(u,v)$ **alors**
         - ⋆ $d[v] \leftarrow d[u] + w(u,v)$
         - ⋆ $\pi[v] \leftarrow u$
3. **Contrôle des cycles de poids négatif :**
   - **Pour chaque** arc $(u,v) \in E$ **faire**
     - **Si** $d[v] > d[u] + w(u,v)$ **alors Retourner** `FAUX` (cycle négatif détecté)
4. **Retourner** `VRAI`

3.2.2.3 C. Complexité

La complexité temporelle est de $\mathcal{O}(|V| \cdot |E|)$, ce qui le rend plus lent que Dijkstra mais plus général.

3.2.3 Illustration des algorithmes de Dijkstra et Bellman-Ford

Considérons le graphe orienté valué $G = (V,E)$ représenté ci-dessous, comportant 5 sommets $V = \{S,A,B,C,D\}$ et le sommet source S :

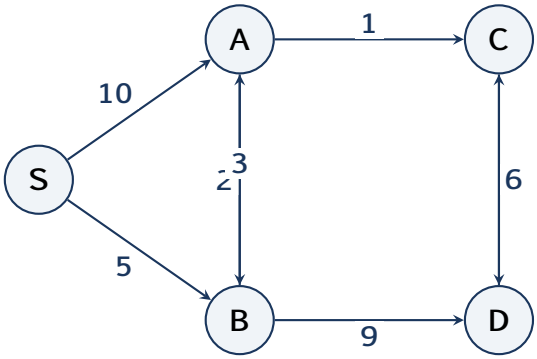


Figure 3.2.1 – Graphe orienté valué positif G.

3.2.3.1 1. Déroulement de l'algorithme de Dijkstra (Source : S)

Étape	Sommet choisi u	Arcs examinés	$d[S]$	$d[A]$	$d[B]$	$d[C]$	$d[D]$
0	Initialisation	–	**0**	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	**S**	$(S,A) : 10$, $(S,B) : 5$	$\underline{0}$	10	**5**	$+\infty$	$+\infty$
2	**B**	$(B,A) : 5+3=8<10$, $(B,D) : 5+9=14$	$\underline{0}$	**8**	$\underline{5}$	$+\infty$	14
3	**A**	(A,B) : ignoré, $(A,C) : 8+1=9$	$\underline{0}$	$\underline{8}$	$\underline{5}$	**9**	14
4	**C**	$(C,D) : 9+4=13<14$	$\underline{0}$	$\underline{8}$	$\underline{5}$	$\underline{9}$	**13**
5	**D**	(D,C) : pas d'amélioration	$\underline{0}$	$\underline{8}$	$\underline{5}$	$\underline{9}$	$\underline{13}$

TABLE 3.2 – Évolution du vecteur de distances d (Dijkstra). Les valeurs soulignées sont définitives.

Résultat des pères (π) : $\pi[S] = \text{NIL}$, $\pi[A] = B$, $\pi[B] = S$, $\pi[C] = A$, $\pi[D] = C$.

3.2.3.2 2. Exécution de Bellman-Ford sur un graphe avec poids négatif

Considérons à présent une modification de l'arc (A,B) avec un poids négatif $w(A,B) = -2$ (sans cycle négatif).

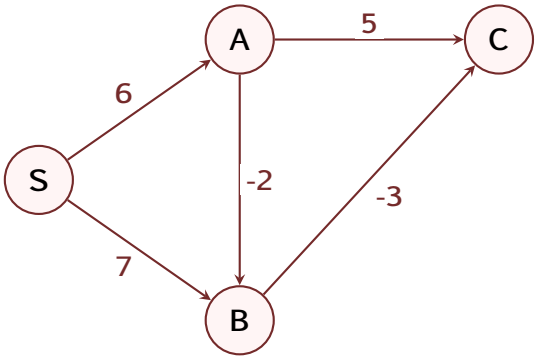


FIGURE 3.2.2 – Graphe avec poids d'arcs négatifs.

Itération	Arcs relâchés	$d[S]$	$d[A]$	$d[B]$	$d[C]$
0	Initialisation	0	$+\infty$	$+\infty$	$+\infty$
1	$(S,A),(S,B),(A,B),(A,C),(B,C)$	0	6	$6+(-2)=4$	$4+(-3)=1$
2	Tous les arcs (aucun changement)	0	6	4	1

TABLE 3.3 – Évolution des distances par Bellman-Ford (stabilisation à l'itération 2).

Arbre du plus court chemin final : $S \to A \to B \to C$ avec une distance totale de $d[C] = 1$.

3.3 Algorithmes pour arbres couvrants : Kruskal, Prim

Soit $G = (V,E)$ un graphe connexe non orienté, où chaque arête $e = (u,v) \in E$ possède un poids $w(e) \in \mathbb{R}$.
Un **arbre couvrant** (*spanning tree*) de G est un sous-graphe $T = (V,E_T)$ avec $E_T \subseteq E$ qui est un arbre (connexe et sans cycle) et qui contient tous les sommets de V. Le problème de l'**Arbre Couvrant de Poids Minimum (ACPM / MST – Minimum Spanning Tree)** consiste à trouver un tel arbre T qui minimise la somme globale des poids de ses arêtes :

$$w(T) = \sum_{e \in E_T} w(e)$$

3.3.1 Algorithme de Kruskal

3.3.1.1 A. Principe et structures Union-Find

L'algorithme de Kruskal adopte une stratégie **gloutonne centrée sur les arêtes** :

1. On trie l'ensemble des arêtes E par ordre de poids croissant.
2. On parcourt les arêtes triées et on ajoute l'arête courante $e = (u,v)$ à l'arbre T si et seulement si elle ne crée pas de cycle avec les arêtes déjà sélectionnées.

Pour détecter efficacement les cycles, on utilise une structure de données pour ensembles disjoints (**Union-Find** / *Disjoint-Set*) proposant trois opérations :

- `Créer-Ensemble(u)` : initialise un ensemble contenant uniquement le sommet u.
- `Trouver-Ensemble(u)` : retourne le représentant du composant contenant u.
- `Union(u, v)` : fusionne les deux composantes connexes de u et v.

3.3.1.2 B. Spécification de l'algorithme

Définition 3.3.1 Algorithme de Kruskal

Entrée : Graphe connexe non orienté $G = (V,E)$ valué par w
Sortie : Ensemble des arêtes E_T formant un ACPM

1. $E_T \leftarrow \emptyset$
2. **Pour chaque** sommet $v \in V$ **faire**
   - `Créer-Ensemble(v)`
3. Trier l'ensemble des arêtes E dans l'ordre croissant des poids w
4. **Pour chaque** arête $e = (u,v) \in E$ (prise dans l'ordre croissant) **faire**
   - **Si** `Trouver-Ensemble(u)` $\neq$ `Trouver-Ensemble(v)` **alors**
     - $E_T \leftarrow E_T \cup \{(u,v)\}$
     - `Union(u, v)`
5. **Retourner** E_T

3.3.1.3 C. Complexité

- Tri des arêtes : $\mathcal{O}(|E|\log|E|)$.
- Opérations Union-Find avec compression de chemin et union par rang : $\mathcal{O}(|E|\cdot\alpha(|V|))$, où α est l'inverse de la fonction d'Ackermann.
- **Complexité globale :** $\mathcal{O}(|E|\log|V|)$.

3.3.2 Algorithme de Prim

3.3.2.1 A. Principe et croissance d'une composante unique

L'algorithme de Prim adopte une stratégie **gloutonne centrée sur les sommets**. Contrairement à Kruskal (qui gère une forêt d'arbres), Prim fait croître **un seul arbre** depuis un sommet racine initial r jusqu'à recouvrir V :

1. On choisit un sommet racine $r \in V$ et on l'ajoute à l'arbre T.
2. À chaque étape, on sélectionne l'arête de poids minimal reliant un sommet appartenant déjà à T à un sommet $v \notin T$.

3.3.2.2 B. Spécification de l'algorithme

Définition 3.3.2 Algorithme de Prim

Entrée : Graphe connexe non orienté $G = (V,E)$ valué par w, sommet racine $r \in V$
Sortie : Arbre des pères π formant un ACPM

1. **Pour chaque** $u \in V$ **faire**
   - $cle[u] \leftarrow +\infty, \quad \pi[u] \leftarrow$ NIL
2. $cle[r] \leftarrow 0$
3. $Q \leftarrow V$ (file de priorité min contenant tous les sommets)
4. **Tant que** $Q \neq \emptyset$ **faire**
   - $u \leftarrow$ Extraire-Min(Q)
   - **Pour chaque** voisin $v \in$ Adjacents$[u]$ **faire**
     - **Si** $v \in Q$ **et** $w(u,v) < cle[v]$ **alors**
       - $\pi[v] \leftarrow u$
       - $cle[v] \leftarrow w(u,v)$
       - Diminuer-Clé($Q,v,cle[v]$)

3.3.2.3 C. Complexité

- Avec un tas binaire : $\mathcal{O}(|E|\log|V|)$.
- Avec un tas de Fibonacci : $\mathcal{O}(|E|+|V|\log|V|)$.

3.3.3 Illustration des algorithmes de Kruskal et Prim

Considérons le graphe non orienté valué $G = (V,E)$ ci-dessous composé de 5 sommets $V = \{A,B,C,D,E\}$:

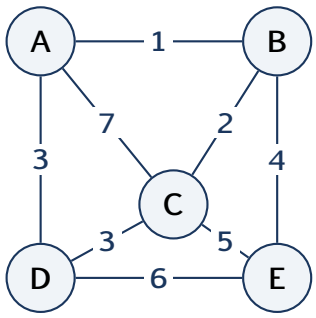


FIGURE 3.3.1 – Graphe pondéré non orienté G.

3.3.3.1 1. Exécution de l'algorithme de Kruskal

1 **Tri des arêtes par poids croissant :**

$$(A,B):1 \quad < \quad (B,C):2 \quad < \quad (A,D):3 \quad = \quad (C,D):3 \quad < \quad (B,E):4 \quad < \quad (C,E):5 \quad < \quad (D,$$

2 **Sélection progressive des arêtes :**

Étape	Arête examinée	Poids	Action	Composantes connexes
1	(A,B)	1	**Acceptée**	$\{A,B\},\{C\},\{D\},\{E\}$
2	(B,C)	2	**Acceptée**	$\{A,B,C\},\{D\},\{E\}$
3	(A,D)	3	**Acceptée**	$\{A,B,C,D\},\{E\}$
4	(C,D)	3	**Rejetée** (Cycle $A-B-C-D-A$)	$\{A,B,C,D\},\{E\}$
5	(B,E)	4	**Acceptée**	$\{A,B,C,D,E\}$ (Terminé)

TABLE 3.4 – Déroulement de l'algorithme de Kruskal.

3.3.3.2 2. Exécution de l'algorithme de Prim (Racine $r = A$)

Étape	Sommet extrait	Mise à jour des clés	Arête retenue	Poids	Sommets dans T
0	Initialisation	$cle[A] = 0$, tous les autres $+\infty$	–	–	$\{A\}$
1	**A**	$cle[B] = 1, cle[D] = 3, cle[C] = 7$	(A,B)	1	$\{A,B\}$
2	**B**	$cle[C] = \min(7,2) = 2, cle[E] = 4$	(B,C)	2	$\{A,B,C\}$
3	**C**	$cle[D] = \min(3,3) = 3, cle[E] = \min(4,5) = 4$	(A,D)	3	$\{A,B,C,D\}$
4	**D**	$cle[E] = \min(4,6) = 4$	(B,E)	4	$\{A,B,C,D,E\}$

TABLE 3.5 – Déroulement de l'algorithme de Prim avec racine A.

3.3.3.3 3. Arbre Couvrant de Poids Minimum (ACPM) obtenu

Les deux algorithmes aboutissent au même arbre couvrant de poids total $\mathbf{W = 1 + 2 + 3 + 4 = 10}$:

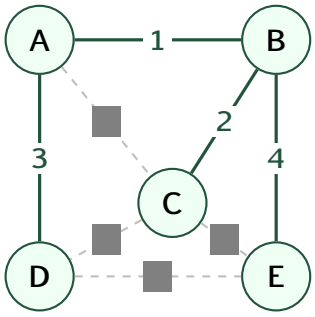


FIGURE 3.3.2 – L'Arbre Couvrant de Poids Minimum (ACPM) retenu.

3.4 Flots dans les réseaux : Algorithme de Ford-Fulkerson

Le problème du flot maximal consiste à déterminer la quantité maximale de matière, d'information ou de trafic pouvant meubler un réseau entre un sommet source et un sommet puit, sans dépasser les capacités limitées des arcs.

3.4.1 Concept de flot et propriétés fondamentales

Définition 3.4.1 Réseau de transport

Un **réseau de transport** est un quadruplet $\mathcal{N} = (G,c,s,t)$ où :

- $G = (V,E)$ est un graphe orienté.
- $c : E \to \mathbb{R}^+$ est la fonction de **capacité** ($c(u,v) \geq 0$).
- $s \in V$ est la **source** (de degré entrant nul).
- $t \in V$ est le **puits** (de degré sortant nul), avec $s \neq t$.

Définition 3.4.2 Flot dans un réseau

Un **flot** dans $\mathcal{N}$ est une fonction $f : E \to \mathbb{R}^+$ vérifiant trois conditions :

1. **Contrainte de capacité :** $\forall (u,v) \in E, \quad 0 \leq f(u,v) \leq c(u,v)$.
2. **Anti-symétrie :** $\forall (u,v) \in E, \quad f(u,v) = -f(v,u)$.
3. **Conservation du flot :** $\forall u \in V \setminus \{s,t\}, \quad \sum_{v \in V} f(v,u) = \sum_{v \in V} f(u,v)$.

La **valeur du flot**, notée $|f|$, est la quantité nette sortant de la source s :

$$|f| = \sum_{v \in V} f(s,v)$$

3.4.2 Algorithme de Ford-Fulkerson

3.4.2.1 A. Réseau résiduel et chemins augmentants

L'algorithme repose sur deux notions fondamentales :

- **Capacité résiduelle :** $c_f(u,v) = c(u,v) - f(u,v)$ pour un arc existant, et $c_f(v,u) = f(u,v)$ pour un arc inverse (possibilité d'annuler du flot).
- **Réseau résiduel** $G_f = (V, E_f)$ **:** Graphe composé des arcs ayant une capacité résiduelle strictement positive ($c_f(u,v) > 0$).
- **Chemin augmentant :** Un chemin orienté de s à t dans le réseau résiduel G_f. La capacité goulot d'étranglement de ce chemin est $c_f(p) = \min\{c_f(u,v) \mid (u,v) \in p\}$.

3.4.2.2 B. Spécification de l'algorithme

Définition 3.4.3 Algorithme de Ford-Fulkerson

Entrée : Réseau $\mathcal{N} = (G,c,s,t)$
Sortie : Flot maximal f et sa valeur $|f|$

1. **Pour chaque** arc $(u,v) \in E$ **faire**
   - $f(u,v) \leftarrow 0$
2. **Tant qu'**il existe un chemin augmentant p de s à t dans le réseau résiduel G_f **faire**
   - $c_f(p) \leftarrow \min\{c_f(u,v) \mid (u,v) \in p\}$
   - **Pour chaque** arc $(u,v) \in p$ **faire**
     - **Si** $(u,v) \in E$ **alors**
       - $f(u,v) \leftarrow f(u,v) + c_f(p)$
     - **Sinon** // Arc inverse
       - $f(v,u) \leftarrow f(v,u) - c_f(p)$
3. **Retourner** f

Théorème 3.4.1 Théorème du Flot Maximal / Coupe Minimale

Pour tout réseau de transport, la valeur maximale d'un flot $s-t$ est égale à la capacité minimale d'une coupe $s-t$.

3.4.2.3 C. Complexité

Si les capacités sont entières, la complexité est $\mathcal{O}(|E| \cdot |f_{\max}|)$. L'algorithme d'Edmonds-Karp (recherche du chemin augmentant par BFS) garantit une complexité polynomiale de $\mathcal{O}(|V| \cdot |E|^2)$.

3.4.3 Illustration de l'algorithme de Ford-Fulkerson

Considérons le réseau ci-dessous avec la source s et le puit t. Les étiquettes sur les arcs représentent la capacité initiale $c(u,v)$.

3.4.3.1 1. Déroulement étape par étape

3.4.3.2 2. Flot Maximal obtenu et Coupe Minimale

À l'étape 4, il n'existe plus aucun chemin de s à t dans le réseau résiduel. La valeur du **flot maximal est $|f_{\max}| = 7 + 8 + 2 = 17$**.

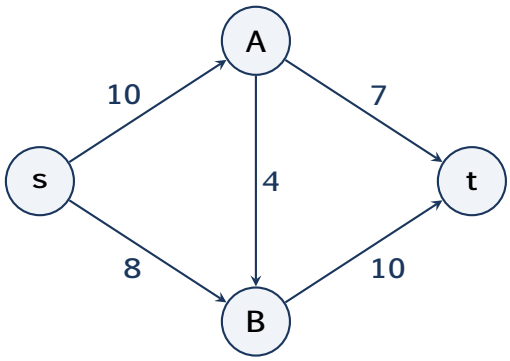


FIGURE 3.4.1 – Réseau de transport initial.

Étape	Chemin augmentant p	Goulot $c_f(p)$	Flot ajouté	Flots sur les arcs f/c
0	Initialisation	0	0	Tous les flots à 0
1	$s \to A \to t$	$\min(10,7) = \mathbf{7}$	+7	$f(s,A) = 7/10$, $f(A,t) = 7/7$
2	$s \to B \to t$	$\min(8,10) = \mathbf{8}$	+8	$f(s,B) = 8/8$, $f(B,t) = 8/10$
3	$s \to A \to B \to t$	$\min(10-7,4,10-8) = \mathbf{2}$	+2	$f(s,A) = 9/10$, $f(A,B) = 2/4$, $f(B,t) = 10/10$

TABLE 3.6 – Trace d'exécution de Ford-Fulkerson.

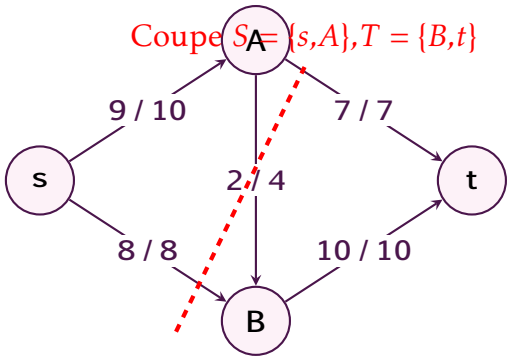


FIGURE 3.4.2 – Réseau final saturé avec les valeurs f/c. La coupe minimale associée a une capacité de $c(s,B) + c(A,t) + c(A,B) = 8 + 7 + 2 = 17$.

Partie II

TRAVAUX DIRIGÉS

Feuille d'exercices n° 1

Introduction à la théorie des graphes

Exercice 1 – *Modélisation et lemmes fondamentaux*

1. Un réseau informatique est composé de 7 serveurs. Est-il possible de connecter ces serveurs de telle sorte que leurs nombres de connexions respectifs soient :
   (a) Cas A : 3,3,3,3,3,3,3 ?
   (b) Cas B : 2,3,4,4,5,5,5 ?
   (c) Cas C : 1,2,2,3,4,4,4 ?

   (Justifier théoriquement à l'aide du Lemme des Poignées de Main).
2. Démontrer que dans tout graphe simple non orienté comportant au moins 2 sommets ($n \geq 2$), il existe au moins deux sommets ayant exactement le même degré.
3. Quelle est la signification concrète de cette propriété dans un réseau d'utilisateurs ou d'équipements informatiques ?

Exercice 2 – *Représentations mémoire et complexité de stockage*

On considère un graphe orienté $G = (V,E)$ représenté en mémoire pour une application réseau, où $n = |V|$ représente le nombre de routeurs et $m = |E|$ le nombre de liaisons directes.

1. Donner la complexité en espace mémoire ($\mathcal{O}$) pour :
   (a) La représentation par **Matrice d'adjacence** A.
   (b) La représentation par **Liste d'adjacence** L.
2. Comparer l'efficacité asymptotique ($\mathcal{O}$) des opérations suivantes selon la structure choisie (A ou L) :
   (a) Tester l'existence d'un arc direct (u,v).
   (b) Calculer le degré entrant $d^-(u)$ d'un sommet u.
   (c) Calculer le degré sortant $d^+(u)$ d'un sommet u.
3. Quel choix de représentation préconisez-vous pour un réseau métropolitain très dense ($m \approx n^2$) ? Pour un réseau WAN très dispersé ($m \approx \mathcal{O}(n)$) ? Justifier votre réponse.

Exercice 3 – *Calcul algébrique sur les matrices d'adjacence*

Soit $G = (V,E)$ un graphe orienté à 4 sommets $V = \{1,2,3,4\}$ représenté par la matrice d'adjacence A suivante :

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

1. Dessiner le graphe orienté G.
2. Déterminer les degrés entrants $d^-(i)$ et sortants $d^+(i)$ pour chaque sommet $i \in \{1,2,3,4\}$.
3. Calculer la matrice A^2, puis la matrice A^3.
4. En déduire le nombre exact de chemins de longueur 3 reliant le sommet 1 au sommet 4. Lister explicitement ces chemins.

Exercice 4 – *Analyse des topologies réseaux usuelles*

On étudie les propriétés structurales (ordre n, nombre d'arêtes m, degré maximal Δ, degré minimal δ et degré moyen) des topologies classiques de réseaux informatiques à n nœuds :

1. **Topologie totalement maillée (K_n) :** Déterminer le degré de chaque sommet, la valeur de $\Delta(K_n)$, de $\delta(K_n)$ et le nombre total d'arêtes.
2. **Topologie en anneau (C_n) :** Donner le degré de chaque sommet et préciser la régularité du graphe.
3. **Topologie en étoile (S_n) :** Comportant 1 switch central connecté à $n-1$ équipements clients, déterminer $\Delta(S_n)$, $\delta(S_n)$ ainsi que le degré moyen des sommets.
4. Si le nombre d'équipements n augmente considérablement, quelle topologie présente le risque le plus élevé de goulot d'étranglement au niveau matériel ?

Exercice 5 – *Degré d'un graphe et analyse matricielle*

Soit $G = (V,E)$ un graphe simple non orienté à n sommets et m arêtes, associé à sa matrice d'adjacence $A = (a_{ij})_{1 \le i,j \le n}$.

1. Exprimer le degré $d(v_i)$ d'un sommet v_i à partir des coefficients a_{ij} de la matrice A.
2. Montrer que la somme de tous les éléments de la matrice A vérifie :

$$\sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij} = 2m$$

3. On considère la matrice d'adjacence suivante :

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Déterminer le degré maximal $\Delta(G)$, le degré minimal $\delta(G)$ et le degré moyen de ce graphe.

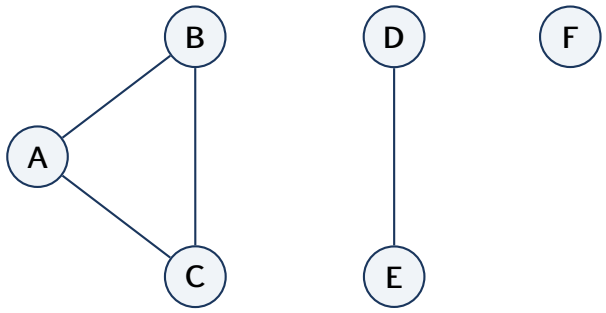
4 Le graphe représenté par cette matrice est-il régulier ? Justifier.

Feuille d'exercices n° 2

Propriétés fondamentales des graphes

Exercice 1 – *Connexité, forte connexité et composantes connexes*

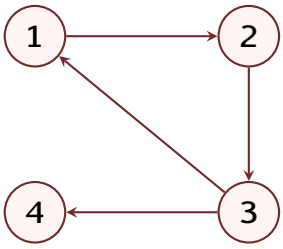
1 Soit un réseau local non orienté $G_1 = (V,E)$ représenté ci-dessous :



(a) Déterminer les composantes connexes de G_1. Le réseau est-il connexe ?

(b) Quel est le nombre minimal d'arêtes à ajouter pour rendre G_1 connexe ?

2 On considère à présent le réseau orienté G_2 suivant :



(a) Déterminer les composantes fortement connexes de G_2.

(b) G_2 est-il fortement connexe ? Justifier.

Exercice 2 – *Parcours eulériens et hamiltoniens dans un réseau de routage*

1 **Graphes eulériens :**

(a) Énoncer le théorème d'Euler (caractérisation par les degrés des sommets).

(b) Un technicien doit inspecter toutes les liaisons physiques d'un réseau sans repasser deux fois par le même câble. Est-il possible de former un circuit eulérien dans un réseau dont la liste des degrés des sommets est : $d(A) = 4, d(B) = 2, d(C) = 3, d(D) = 3, d(E) = 2$? Et une chaîne eulérienne ?

2 **Graphes hamiltoniens :**

(a) Un paquet informatique doit visiter chaque routeur d'un réseau une et une seule fois avant de revenir à son point d'origine.

(b) Rappeler le théorème de Dirac et le théorème de Ore.

(c) Un graphe simple à 6 sommets où le degré de chaque sommet est supérieur ou égal à 3 admet-il nécessairement un cycle hamiltonien ? Justifier.

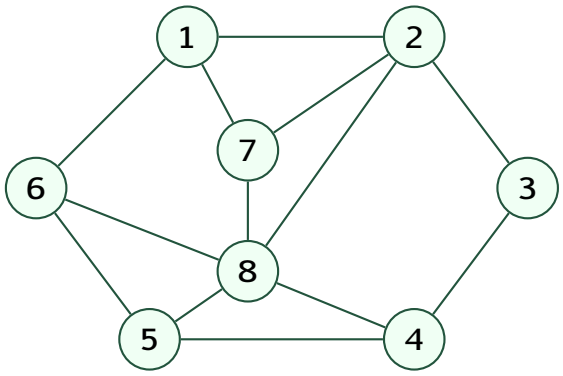
Exercice 3 – *Arbres, arborescences et planarité*

1 **Propriétés des arbres :** Démontrer par récurrence que tout arbre à n sommets possède exactement $n-1$ arêtes.

2 **Planarité :**

(a) Rappeler la formule d'Euler pour un graphe connexe plan ($S - A + F = 2$).

(b) On considère le graphe plan représenté ci-dessous :

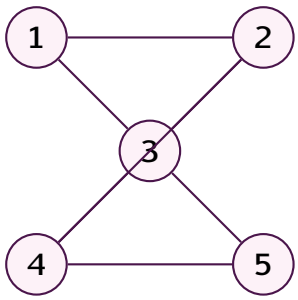


Vérifier la formule d'Euler en décomptant le nombre de sommets, d'arêtes et de faces (régions).

(c) Démontrer que le graphe complet K_5 n'est pas plan à l'aide de l'inégalité $A \leq 3S - 6$.

Exercice 4 – *Coloriage de graphes et allocation de fréquences radio*

Un opérateur télécom doit attribuer des fréquences radio à 5 antennes. Deux antennes ayant une zone d'interférence doivent utiliser des fréquences différentes (couleurs différentes). Le graphe des interférences est le suivant :



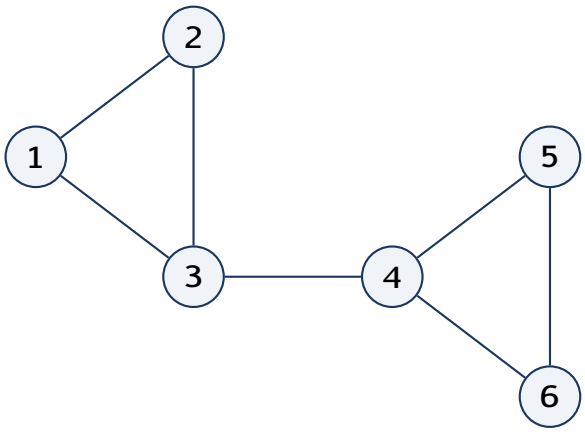
1 Déterminer le degré de chaque sommet.

2 Déterminer le nombre chromatique $\chi(G)$ de ce graphe.

3 Appliquer l'algorithme glouton (Welsh-Powell) en ordonnant les sommets par degré décroissant pour attribuer un nombre minimal de fréquences.

Exercice 5 – *Cliques et ensembles stables*

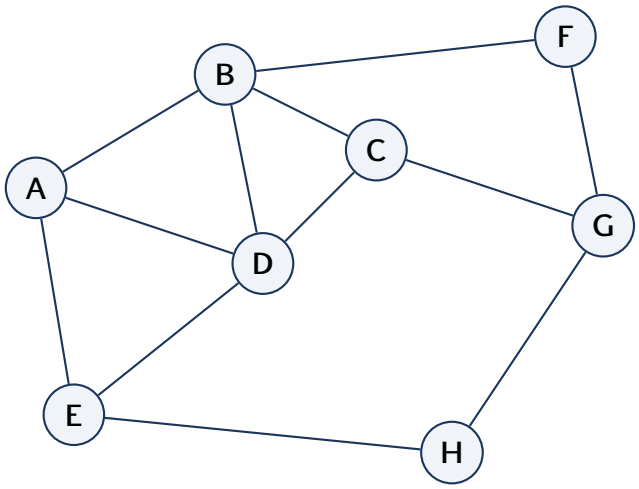
Soit le graphe simple non orienté G à 6 sommets représenté ci-dessous :



1. Définir formellement une **clique** et un **ensemble stable** dans un graphe.
2. Déterminer l'ordre de la plus grande clique (nombre de clique $\omega(G)$).
3. Déterminer la taille du plus grand ensemble stable (nombre de stabilité $\alpha(G)$).
4. Quelle relation existe-t-il entre un ensemble stable dans G et une clique dans le graphe complémentaire $\bar{G}$?

Exercice 6 – *Routage et redondance dans un réseau informatique d'entreprise*

Une entreprise de services numériques déploie un réseau local interconnectant 8 commutateurs principaux nommés A,B,C,D,E,F,G,H. Les liaisons physiques en fibre optique permettent un transfert de données bidirectionnel entre certains commutateurs.
La topologie du réseau est représentée par le graphe non orienté G ci-dessous :



1. (a) Déterminer, en justifiant, si la topologie du réseau est totalement maillée (graphe complet).
   (b) Déterminer, en justifiant, si tout commutateur peut communiquer directement ou indirectement avec n'importe quel autre commutateur (graphe connexe).

2 Un technicien réseau doit tester l'intégralité des liaisons physiques en passant une et une seule fois par chaque câble optique. Déterminer, en justifiant, si le graphe G admet une chaîne eulérienne. Si oui, donner une telle chaîne.

3 Donner la matrice d'adjacence M du graphe G en respectant l'ordre alphabétique des sommets.

4 Un paquet de données doit être acheminé du commutateur B au commutateur H.

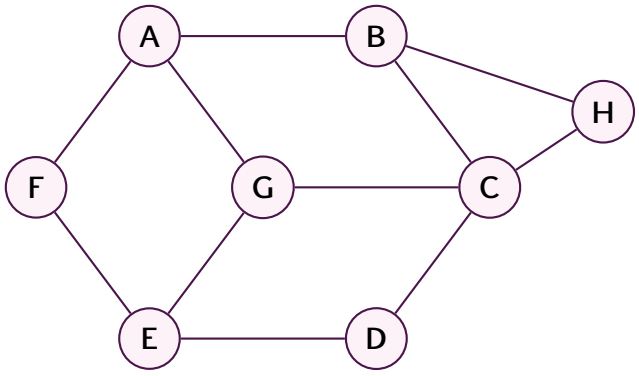
(a) Déterminer le nombre minimal de sauts (liaisons) qu'il doit prendre à l'aide des matrices M^2 et M^3.

(b) Donner tous les trajets possibles empruntant trois liaisons successives.

(c) Est-il vrai que les commutateurs quelconques peuvent toujours être reliés par trois sauts ou moins ? Justifier.

Exercice 7 – *Optimisation d'une tournée de collecte de déchets électroniques*

Dans le cadre d'un projet Smart City, une municipalité gère 8 points de collecte de déchets électroniques désignés par les stations A,B,C,D,E,F,G,H. Les axes routiers autorisés aux véhicules de collecte permettent la circulation dans les deux sens entre certaines stations adjacentes.
Le réseau routier et l'emplacement des stations sont modélisés par le graphe ci-dessous :



1 (a) Déterminer, en justifiant, si le graphe G est complet.

(b) Déterminer, en justifiant, si le graphe G est connexe.

2 Le camion de collecte doit parcourir tous les axes routiers du réseau sans jamais emprunter deux fois le même tronçon. Déterminer, en justifiant, s'il existe un tel itinéraire (chaîne eulérienne) et le préciser si besoin.

3 Donner la matrice d'adjacence M du graphe G selon l'ordre alphabétique des sommets.

4 Le responsable logistique souhaite envoyer un véhicule de la station F à la station H.

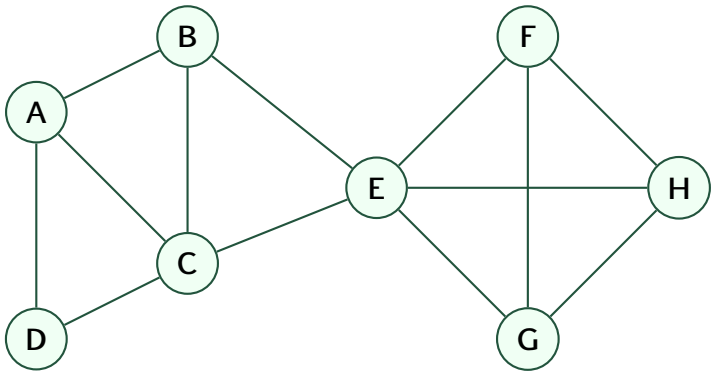
(a) Déterminer le nombre minimal de tronçons routiers à parcourir. Justifier à l'aide des puissances de la matrice M.

(b) Lister tous les itinéraires possibles constitués de 3 tronçons successifs.

(c) Est-il vrai que deux stations quelconques peuvent toujours être reliées par 3 tronçons ou moins ? Justifier.

Exercice 8 – *Architecture de distribution d'eau potable*

Une société de distribution d'eau potable gère une station régionale comportant 8 vannes principales d'interconnexion notées A,B,C,D,E,F,G,H. Les conduites sous pression permettent un écoulement bidirectionnel de l'eau entre les vannes interconnectées.
La topologie du réseau de conduites est modélisée par le graphe ci-dessous :



1 (a) Déterminer, en justifiant, si la structure du réseau d'eau est complètement interconnectée (graphe complet).

(b) Déterminer, en justifiant, si l'eau peut circuler entre n'importe quelle paire de vannes du réseau (graphe connexe).

2 Un robot d'inspection doit parcourir chaque conduite du réseau exactement une fois pour détecter les fuites. Déterminer si le graphe G admet une chaîne eulérienne. Si oui, proposer un itinéraire d'inspection complet.

3 Donner la matrice d'adjacence M associée à ce réseau d'eau en respectant l'ordre alphabétique des sommets.

4 On cherche à acheminer un débit d'eau de la vanne A vers la vanne H.

(a) Déterminer le nombre minimal de conduites à traverser à l'aide des matrices M^2 et M^3.

(b) Donner tous les parcours possibles empruntant trois conduites successives.

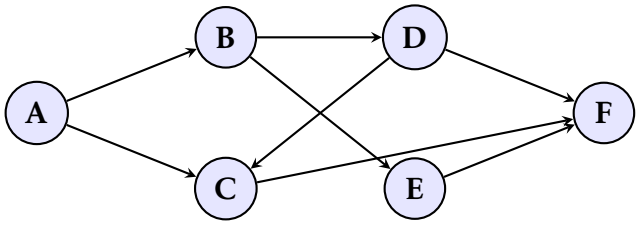
(c) Est-il vrai que deux vannes quelconques du réseau peuvent toujours être reliées en traversant au plus trois conduites ? Justifier la réponse.

Feuille d'exercices n° 3

Les algorithmes sur les graphes

Exercice 1 – *Parcours de graphes et classification des arcs (BFS et DFS)*

On considère le réseau informatique représenté par le graphe orienté $G = (V, E)$ ci-dessous, où les sommets représentent des routeurs :



1 **Parcours en largeur (BFS) :**

(a) Donner l'ordre de découverte des sommets lors d'un parcours BFS en partant du sommet source A. *(En cas d'égalité de priorité, respecter l'ordre alphabétique).*

(b) Lister les arêtes formant l'arbre couvrant BFS.

(c) Déterminer la distance (nombre minimal d'arcs) reliant le sommet source A à chacun des autres sommets du réseau.

2 **Parcours en profondeur (DFS) :**

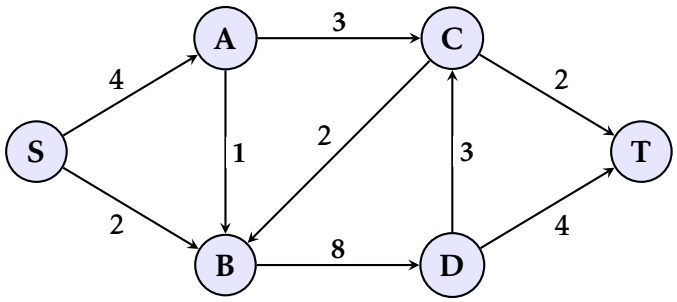
(a) Exécuter l'algorithme DFS à partir du sommet source A en indiquant pour chaque sommet ses dates de début et de fin de traitement sous la forme $d[v]/f[v]$.

(b) Classer l'ensemble des arcs du graphe selon les quatre catégories standard du DFS : **arcs d'arbre**, **arcs arrières**, **arcs avant** et **arcs croisés**.

3 **Propriétés algorithmiques :** Donner la complexité temporelle et spatiale théorique des algorithmes BFS et DFS lorsque le graphe est représenté par une liste d'adjacence.

Exercice 2 – *Plus courts chemins et routage dynamique (Dijkstra et Bellman-Ford)*

Un réseau d'entreprise interconnecte 6 routeurs $\{S, A, B, C, D, T\}$. Les latences des liaisons (en ms) sont représentées par les poids des arcs dans le graphe orienté ci-dessous :



1 **Algorithme de Dijkstra :**

(a) Énoncer les conditions d'application de l'algorithme de Dijkstra.

(b) Exécuter l'algorithme de Dijkstra à partir du sommet source S en consignant les étapes dans un tableau de résolution (étiquettes des distances et sommets fixes).

(c) Extraire la longueur minimale du trajet reliant S à T ainsi que la séquence exacte des routeurs traversés.

2 **Algorithme de Bellman-Ford et cycles négatifs :**

(a) Quel est l'intérêt principal de l'algorithme de Bellman-Ford par rapport à Dijkstra dans un protocole de routage ?

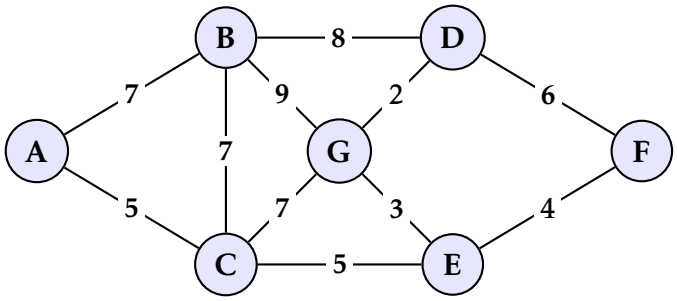
(b) On modifie le poids de l'arc (B,D) tel que $w(B,D) = -5$. Dérouler l'algorithme de Bellman-Ford (étapes de relaxation des arcs) à partir du sommet source S.

(c) Détecter si cette modification introduit un cycle de poids négatif. Justifier formellement la condition d'arrêt/détection.

3 **Complexité :** Exprimer et comparer la complexité temporelle théorique de Dijkstra (avec tas binaire) et de Bellman-Ford pour un graphe à n sommets et m arcs.

Exercice 3 – *Optimisation d'infrastructures et arbres couvrants minimaux (Kruskal et Prim)*

Une entreprise souhaite interconnecter 7 centres de données $\{A,B,C,D,E,F,G\}$ par de la fibre optique à moindre coût. Les coûts de déploiement (en dizaines de milliers d'euros) sont modélisés par le graphe non orienté pondéré ci-dessous :



1 **Algorithme de Kruskal et structure Union-Find :**

(a) Énoncer le principe de l'algorithme de Kruskal et expliquer le rôle de la structure de données des ensembles disjoints (Union-Find) dans la prévention des cycles.

(b) Dresser la liste triée de toutes les arêtes du graphe par ordre croissant de leurs poids.

(c) Dérouler l'algorithme de Kruskal en indiquant clairement pour chaque arête si elle est retenue ou rejetée (avec le motif du rejet).

2 **Algorithme de Prim :**

(a) Expliquer la différence de stratégie de construction entre l'algorithme de Prim et l'algorithme de Kruskal.

(b) Dérouler l'algorithme de Prim à partir du sommet source A, en précisant à chaque étape l'arête sélectionnée, son poids, et la nouvelle composante en croissance.

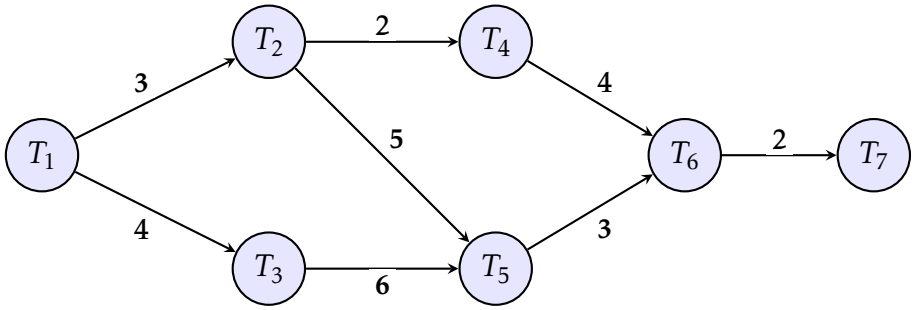
3 **Analyse des résultats :**

(a) Calculer le poids total de l'arbre couvrant minimal (ACM) obtenu.

(b) L'arbre couvrant minimal d'un graphe est-il toujours unique ? Justifier la réponse au regard des poids des arêtes de ce graphe.

Exercice 4 – *Analyse de dépendances et chemin critique dans un DAG*

Un projet de déploiement réseau est découpé en 7 tâches $\{T_1, T_2, T_3, T_4, T_5, T_6, T_7\}$. Le graphe orienté acyclique (DAG) ci-dessous modélise la séquence d'exécution : un arc (T_i, T_j) de poids w indique une contrainte de précédence où T_i requiert w jours d'exécution avant que T_j ne puisse débuter.



1 **Tri topologique et ordonnancement :**

(a) Définir formellement un tri topologique et rappeler la condition nécessaire et suffisante d'existence d'un tel tri dans un graphe orienté.

(b) Obtenir un ordonnancement valide des tâches en appliquant l'algorithme du tri topologique basé sur le parcours en profondeur (DFS).

2 **Calcul des dates et du chemin critique :**

(a) Calculer pour chaque tâche la date au plus tôt (ES_i) et la date au plus tard (LS_i).

(b) Déterminer la durée minimale indispensable pour réaliser la totalité du projet.

(c) Identifier le chemin critique du projet et expliciter les tâches dont le retard impacterait directement la livraison finale.

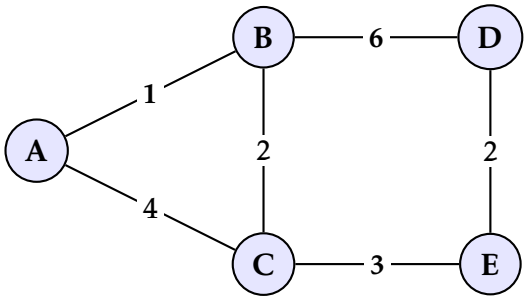
3 **Marge et sensibilité aux retards :**

(a) Calculer la marge totale de la tâche T_4.

(b) Si la réalisation de la tâche T_2 subit un retard imprévu de 2 jours, la date de livraison finale du projet sera-t-elle modifiée ? Justifier formellement.

Exercice 5 – *Routage distribué, vecteur de distances et tolérance aux pannes*

Un sous-réseau régional est constitué de 5 routeurs $\{A, B, C, D, E\}$ interconnectés par des liaisons point-à-point modélisées par le graphe pondéré non orienté ci-dessous. Le réseau utilise un protocole de routage à vecteur de distances basé sur l'algorithme de Bellman-Ford distribué.



1 **Convergence des vecteurs de distances :**

(a) Donner l'état initial du vecteur de distances $D^A = (d(A,A),d(A,B),d(A,C),d(A,D),d(A,E))$ mesuré directement par le routeur A à l'étape $t = 0$.

(b) Énoncer la formule de mise à jour de Bellman-Ford permettant au routeur A de réévaluer la distance $d(A,Y)$ vers une destination Y connaissant les estimations $d(V,Y)$ transmises par ses voisins $V \in \{B,C\}$.

(c) Dresser le tableau des étapes successives d'échange d'informations jusqu'à la convergence globale des tables de routage de l'ensemble des 5 routeurs.

2 **Gestion des pannes et comptage à l'infini :**

(a) La liaison directe $\{A,B\}$ subit une rupture physique (son coût devient infini : $w(A,B) = \infty$). Expliquer comment les estimations de coût vers B dérivent progressivement.

(b) Expliciter le phénomène de **comptage à l'infini** (*Count-to-Infinity*) et ses conséquences sur la saturation du réseau.

(c) Définir et expliquer le principe des mécanismes de prévention **Split Horizon** (horizon divisé) et **Poison Reverse** (empoisonnement de route).

3 **Comparaison synthétique :** Présenter sous forme de tableau comparatif les caractéristiques essentielles des algorithmes de routage à **Vecteur de distances** (ex : RIP) et à **État de liens** (ex : OSPF), selon les critères suivants : complexité temporelle, volume de trafic de contrôle échangé et vitesse de convergence en cas de panne.

Table des matières

Sommaire **1**

PartieI – COURS

1 Introduction à la théorie des graphes **3**
1.1 Définitions et premiers exemples 3
1.1.1 Notion d'adjacence entre sommets 4
1.1.2 Notion de degré d'un sommet 4
1.2 Représentation des graphes 8
1.2.1 Matrice d'adjacence 8
1.2.2 Liste d'adjacence 10
1.2.3 Degré d'un sommet 11
1.2.4 Degré d'un graphe 12

2 Propriétés fondamentales des graphes **13**
2.1 Connexité : composantes connexes, graphes fortement connexes 13
2.1.1 Cas des graphes non orientés 13
2.1.2 Cas des graphes orientés 14
2.2 Graphes eulériens 15
2.3 Graphe hamiltonien 16
2.4 Arbre et arborescence 17
2.4.1 Planarité et coloriage de graphes 18
2.5 Coloriage de graphes, cliques et stables 19
2.5.1 Cliques et ensembles stables 20

3 Les algorithmes sur les graphes **22**
3.1 Parcours de graphes : parcours en largeur (BFS) et en profondeur (DFS) . . 22
3.1.1 Parcours en largeur (BFS – Breadth-First Search) 22
3.1.1.1 A. Principe et fondements théoriques 22
3.1.1.2 B. Spécification de l'algorithme 22
3.1.1.3 C. Propriétés et Complexité 23
3.1.2 Parcours en profondeur (DFS – Depth-First Search) 23
3.1.2.1 A. Principe et fondements théoriques 23
3.1.2.2 B. Spécification de l'algorithme 23
3.1.2.3 C. Classification des arêtes 24
3.1.3 Illustration des parcours 24
3.1.3.1 1. Déroulement détaillé du BFS (Sommet source : A) 24
3.1.3.2 2. Déroulement détaillé du DFS (Sommet source : A) . . . 24
3.2 Algorithmes de recherche de chemins : Dijkstra, Bellman-Ford 25
3.2.1 Algorithme de Dijkstra 25

3.2.1.1 A. Principe et conditions d'application 25
3.2.1.2 B. Spécification de l'algorithme 27
3.2.1.3 C. Complexité 27
3.2.2 Algorithme de Bellman-Ford 27
3.2.2.1 A. Principe et détection de cycles négatifs 27
3.2.2.2 B. Spécification de l'algorithme 28
3.2.2.3 C. Complexité 28
3.2.3 Illustration des algorithmes de Dijkstra et Bellman-Ford 28
3.2.3.1 1. Déroulement de l'algorithme de Dijkstra (Source : S) . . 29
3.2.3.2 2. Exécution de Bellman-Ford sur un graphe avec poids négatif 29
3.3 Algorithmes pour arbres couvrants : Kruskal, Prim 30
3.3.1 Algorithme de Kruskal 30
3.3.1.1 A. Principe et structures Union-Find 30
3.3.1.2 B. Spécification de l'algorithme 30
3.3.1.3 C. Complexité 31
3.3.2 Algorithme de Prim 31
3.3.2.1 A. Principe et croissance d'une composante unique 31
3.3.2.2 B. Spécification de l'algorithme 31
3.3.2.3 C. Complexité 31
3.3.3 Illustration des algorithmes de Kruskal et Prim 31
3.3.3.1 1. Exécution de l'algorithme de Kruskal 32
3.3.3.2 2. Exécution de l'algorithme de Prim (Racine $r = A$) 32
3.3.3.3 3. Arbre Couvrant de Poids Minimum (ACPM) obtenu . . 33
3.4 Flots dans les réseaux : Algorithme de Ford-Fulkerson 33
3.4.1 Concept de flot et propriétés fondamentales 33
3.4.2 Algorithme de Ford-Fulkerson 33
3.4.2.1 A. Réseau résiduel et chemins augmentants 33
3.4.2.2 B. Spécification de l'algorithme 34
3.4.2.3 C. Complexité 34
3.4.3 Illustration de l'algorithme de Ford-Fulkerson 34
3.4.3.1 1. Déroulement étape par étape 34
3.4.3.2 2. Flot Maximal obtenu et Coupe Minimale 34

PartieII – TRAVAUX DIRIGÉS

1 Introduction à la théorie des graphes **37**

2 Propriétés fondamentales des graphes **40**

3 Les algorithmes sur les graphes **45**